Reinforcement Twinning: Towards a Bidirectional Learning Framework for Model Updating and Policy Optimization

Miguel A. Mendez*, Lorenzo Schena, Yannick Lecomte,
Sebastiano Randino, Romain Poletti
von Karman Institute for Fluid Dynamics

February 2026

Reinforcement Twinning (RT) is a cyber–physical learning framework that formalizes a reciprocal learning process in which a digital twin and a control policy co-evolve, continuously reinforcing one another through interaction with the real system. These notes briefly review the structural symmetry between adjoint-based model-based policy search and actor–critic reinforcement learning, linking the Hamiltonian formulation of the former to the Bellman value formulation of the latter.

This symmetry is used to develop mechanisms for reciprocal reinforcement: the digital twin improves the policy by providing structured predictions and sensitivity information, while the policy actively excites and probes the system to refine the twin. Learning thus becomes bidirectional — control informs modeling, and modeling stabilizes and accelerates policy search. The resulting architecture bridges modeling-to-control and control-to-modeling regimes within a unified framework.

A floating offshore wind turbine case study is presented as a proof of concept, illustrating the integration of twinning, model-based control, and reinforcement learning within the proposed architecture. The example provides a concrete demonstration of how reciprocal reinforcement can be implemented in practice.

*mendez@vki.ac.be

Contents

List of Symbols

State-Space Dynamics and Environment

$\boldsymbol{s}_{\bullet,k} \in \mathbb{R}^{n_s}$	Real (black-box) system state at time step k
$\boldsymbol{o}_{\bullet,k} \in \mathbb{R}^{n_o}$	Real system observation
$\boldsymbol{s}_{\circ,k}$	Digital twin (white-box) state
$\boldsymbol{o}_{\circ,k}$	Digital twin observation
$\boldsymbol{a}_k \in \mathbb{R}^{n_a}$	Control input applied to the system
$\boldsymbol{z}_k \in \mathbb{R}^{n_z}$	Exogenous input / disturbance
$F_\bullet(\cdot)$	Real system discrete-time one-step flow map
$h_\bullet(\cdot)$	Real observation operator
$F_\circ(\cdot)$	Digital twin discrete-time one-step flow map
$h_\circ(\cdot)$	Digital twin observation operator

Notation for Learning and Control Framework

$\boldsymbol{p}_k$	Closure variable representing unresolved effects
$g(\cdot \mid \boldsymbol{\theta}_p)$	Parametric closure model
$\boldsymbol{\theta}_p$	Closure model parameters
$\star \in \{\bullet, \circ\}$	Policy index (model-free or model-based)
$\pi_\star(\cdot \mid \boldsymbol{\theta}_{\pi,\star})$	Control policy ($\star \in \{\bullet, \circ\}$)
$\boldsymbol{\theta}_{\pi,\star}$	Policy parameters
$\boldsymbol{a}_{\star,k}$	Action proposed by policy $\pi_\star$
$\boldsymbol{s}_{\circ\star,k}$	Twin state under rollout of policy $\pi_\star$
$\Pi_{(e)}(\cdot)$	Referee operator selecting live policy
$\mathcal{I}_{(e)}$	Supervisory information at episode e
n_t	Time steps per episode
N_e	Number of episodes
N_{tot}	Total real transitions collected
$\mathcal{J}_T^{(e)}$	Digital twin identification cost
$\ell_{\text{pred}}(\cdot)$	Prediction error loss
$\mathcal{Z}$	Disturbance distribution
$\mathcal{U}_p(\cdot)$	Twin parameter update operator
$\mathcal{U}_{\pi,\star}(\cdot)$	Policy parameter update operator
$\mathcal{D}_T$	Identification dataset
$\mathcal{D}_\star$	Transition dataset associated with policy $\pi_\star$
$r(\boldsymbol{s}, \boldsymbol{a})$	Stage reward function
$r_{\star,k}$	Reward at time step k
$\mathcal{R}_{A,\star}$	Cumulative reward of policy $\pi_\star$

Additional Notation for Advanced Reinforcement Twinning Schemes

$Q(\boldsymbol{o}, \boldsymbol{a}; \boldsymbol{\theta}_Q)$	Critic (state–action value function approximator)
$\boldsymbol{\theta}_Q,\ \boldsymbol{\theta}_Q^{\text{targ}}$	Critic parameters and target-network parameters
$\boldsymbol{\theta}_{\pi,\bullet}^{\text{targ}}$	Target-network actor parameters (model-free)
$\mathcal{L}_Q$	Critic loss (Bellman residual)
y_k	Bellman target at time step k
γ	Discount factor
τ	Target-network update rate ($0 < \tau \ll 1$)
N	Mini-batch size
$\boldsymbol{\varepsilon}_k$	Exploration noise injected in the action
$\mathcal{B}$	Replay buffer
$d^\pi(\boldsymbol{s}, \boldsymbol{a})$	State–action distribution induced by policy π
$\boldsymbol{\lambda}_{\circ,k}$	Discrete adjoint variable at time step k
$\mathcal{H}_k$	Discrete Hamiltonian at time step k
H_k	Alternative notation for Hamiltonian
η	Hybrid gradient mixing coefficient ($\eta \in [0, 1]$)
ΔQ_k	Value gap between candidate policies at time k
ζ_k	Confidence-based blending weight in arbitration
$\Pi_k(\cdot)$	Step-wise referee operator
$\mu(\cdot),\ \kappa(\cdot, \cdot)$	Gaussian-process mean and kernel

Floating Wind Turbine Test Case

$R,\ H_t$	Rotor radius and hub height
$\theta,\ \omega = \dot{\theta}$	Platform pitch angle and rate
Ω	Rotor angular velocity
$v_\infty,\ v = v_\infty - H_t\omega$	Free-stream and effective inflow velocity
$T_{\text{aero}},\ \tau_{\text{aero}},\ \tau_g$	Aerodynamic thrust, torque, and generator torque
β	Blade pitch angle
$J_{\text{p}},\ J_{\text{rot}}$	Platform and rotor inertia
$D,\ D_v,\ K$	Linear damping, quadratic damping, restoring stiffness
τ_{waves}	Wave-induced torque
$\rho,\ A = \pi R^2$	Air density and rotor swept area
$C_p(\xi, \beta),\ C_T(\xi, \beta)$	Power and thrust coefficients
ξ	Tip-speed ratio
$\tau_\beta,\ \tau_{\tau_g}$	Actuator time constants
$\sigma,\ \ell$	Kernel amplitude and length scale
T_w	Dominant wave period
$P = \tau_g\Omega,\ P_{\text{ref}}$	Electrical power and reference power
$\boldsymbol{s}_0,\ \bar{\boldsymbol{s}},$	Initial state and equilibrium state
$w_P,\ w_{\Delta P},\ w_\omega$	Reward weighting coefficients
$\boldsymbol{\phi}_k,\ \mathbf{K}$	Policy features and parameter matrix
$\boldsymbol{a}_{\text{mid}},\ \boldsymbol{a}_{\text{amp}}$	Actuation offset and scaling vectors

1 Engineering in the Age of Real-World Data

Over the past two decades, engineered systems have entered what may reasonably be described as an age of real-world data. Advances in sensing, embedded electronics, wireless communication, and Internet-of-Things (IoT) infrastructures have dramatically increased our ability to observe physical processes in situ and in real time (Lee, 2008; Lee and Seshia, 2015; Kagermann et al., 2013; Ficili et al., 2025). Modern industrial assets, energy systems, transportation platforms, manufacturing lines, and environmental infrastructures are densely instrumented with sensor networks that continuously stream measurements of temperature, pressure, vibration, flow, position, and operational states.

Surveys of IoT and cyber-physical systems document the rapid proliferation of connected devices. Industrial IoT emphasizes real-time monitoring, predictive maintenance, and data-driven optimization as core features of current engineering practice (Atzori et al., 2010; Gubbi et al., 2013; Sicari et al., 2015; Tao et al., 2019). Systems once sparsely measured are now continuously observed at high spatial and temporal resolution.

The central question is how to leverage this observational richness. A data-rich environment has the potential to reshape the entire engineering workflow—from design and modeling to monitoring, diagnosis, and control.

This expanded sensing capability arrives at a moment when engineering systems are being pushed into increasingly demanding operational regimes, under off-design, uncertain, and rapidly evolving conditions. For example, offshore wind farms on floating platforms are exposed to wave-induced loads, and atmospheric variability (Veers et al., 2019; Lackner, 2009). Unmanned aerial vehicles must maneuver reliably in disturbed and cluttered environments with rapidly changing flow conditions (Kumar and Michael, 2012). Emerging green propulsion systems, including cryogenic storage and transport technologies, operate under large thermal and dynamical excursions that challenge traditional design assumptions (Adler and Martins, 2023). These examples, drawn primarily from thermofluid systems, illustrate a broader trend. Similar challenges arise across power grids integrating intermittent renewables, electrochemical storage systems subject to degradation, chemical process plants operating under fluctuating feed conditions, or autonomous systems navigating uncertain environments to mention examples from other fields.

In all these engineering systems, the dominant physical processes can be described by first-principles models derived from conservation laws and constitutive relations. However, high-fidelity simulations of these processes are typically too computationally expensive for operational decision-making and real-time flow control. Fast reduced-order models are needed. Among the available options, the most effective are those that retain as much physical structure as possible, embedding conservation principles together with phenomenological coefficients. Examples include aerodynamic load models, damping terms, turbulence closures, reaction rates, or heat- and mass-transfer coefficients (see Mendez et al. (2025)). Yet these coefficients are often derived from empirical correlations obtained under laboratory conditions that may differ significantly from operational regimes. Regardless of the approach, the predictive performance of any reduced-order model deteriorates far from the conditions in which it is derived. This raises a central challenge: how can predictive models be updated or adapted online so that control decisions remain both physically consistent and operationally effective?

The technical means to address this challenge largely exist but a reorganization of

how modeling and control interact. Traditionally, models are built offline, identification is a preliminary step, and control laws assume fixed predictive structures– modeling, estimation, and control are arranged sequentially. Modern cyber–physical systems instead require architectures where these components evolve together within a shared feedback loop. This shift in the engineering paradigm has brought the concept of the digital twin to prominence.

In its most general formulation, a digital twin is a virtual representation of a physical asset that is continuously updated using real-time data and intended to support monitoring, prediction, and decision-making throughout the asset's lifecycle (Tao et al., 2019; Jones et al., 2020; Wagg et al., 2025). Unlike traditional simulation models, which are typically constructed offline and used for scenario analysis, a digital twin is meant to remain dynamically coupled to its physical counterpart during operation. Crucially, the twin is not merely descriptive: as emphasized in recent analyses of the concept (Korenhof et al., 2021; Wagg et al., 2025), it must become an acting—or steering—representation, capable of taking decisions and influence the evolution of the physical process itself. This closes the loop between representation and intervention: The digital model becomes an active element of the closed-loop system, and the real system becomes a source of epistemic gain. The twin can drive control actions and control actions can improve the twin.

To operationalize this bidirectional coupling, model adaptation and control improvement must be coordinated within a unified learning architecture. Reinforcement twinning, first introduced in (Schena et al., 2024) and developed within the ERC project "Re-Twist", formalizes this idea by embedding policy optimization and model refinement within a unified feedback structure.

The following sections define reinforcement twinning (Section 2) and examine its conceptual roots (Section3).

2 What is Reinforcement Twinning?

Reinforcement twinning is a cyber-physical learning architecture in which a digital twin and a control agent operate as a coupled unit around a physical system. Its defining feature is the simultaneous adaptation of the predictive model and the control policy within the same feedback loop.

Figure 1 illustrates the reinforcement twinning (RT) unit and its interaction with the real system. The RT unit is composed of two adaptive components: a digital twin and an acting agent, persistently coupled to a physical system. Their role is described below.

The Real System. The real system represents the physical process or engineering asset under consideration. It evolves according to unknown or partially known dynamics and produces real-time measurements. These measurements enable model–data alignment within the twin and provide performance feedback—such as reward, cost, constraint violation, or task completion signals—to the agent. The system receives real actions from the agent, which determine its trajectory and the operating regimes that are explored.

The Digital Twin. The digital twin offers a predictive model of the real system. It embodies a closure structure or model class whose parameters are tuned to match observed behavior, aiming to improve predictive accuracy. The twin generates internal predictions of state evolution and system response, which the agent can use for simulation, planning,

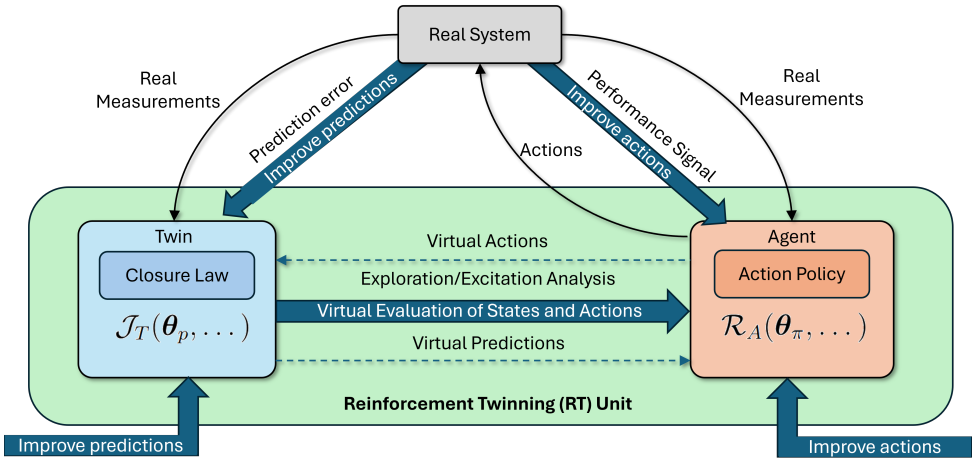


Figure 1: Schematic overview of the *Reinforcement Twinning* (RT) framework. The reinforcement twinning unit links a digital twin and an acting agent to a real system. The real system provides two feedback signals: measurement data for correcting the model and performance signals for improving the policy. The agent queries the twin to evaluate candidate actions, while its chosen actions shape the system's operating conditions and thus the data available to update the twin.

and counterfactual analysis.

The Acting Agent. The acting agent implements a policy that selects actions applied to the real system. Its objective is to improve performance according to a specified criterion. The agent receives performance feedback from the system and may exploit the twin's predictions for virtual evaluation of candidate actions.

Two limiting operating regimes can be distinguished within this architecture, illustrated in Figures 2 and 3. In the *modeling-to-control* regime (Figure 2), the digital twin primarily serves the control agent: predictive simulations and model updates are exploited to improve control performance. This configuration encompasses established methodologies from adaptive control, model predictive control, and model-based reinforcement learning. In the *control-to-modeling* regime (Figure 3), the emphasis is reversed: the agent seek to excite the system to generate more informative data, thereby accelerating model refinement. This regime connects to dual control, active system identification, and information-driven exploration strategies. These conceptual roots are analyzed in more detail in Sections 3.1 and 3.2.

Multi-Level Learning Feedback. Beyond direct interaction between the agent and physical plant, Reinforcement Twinning adds multi-level learning channels. Internally, the twin and agent form a virtual feedback loop: the agent evaluates states and actions with the twin's predictive model, while ongoing twin updates reshape the policy's optimization landscape. This internal coupling complements physical feedback and enables simultaneous policy optimization and model refinement within the RT unit. Externally, an RT unit can run alone or within a network of twinning systems. In such networks, feedback spans units via shared model updates (Marques et al., 2024), shared policies, or aggregated performance signals. This distributed coupling lets reinforcement twinning scale from a single cyber–physical system to fleets of interacting assets, making learning both local and collective.

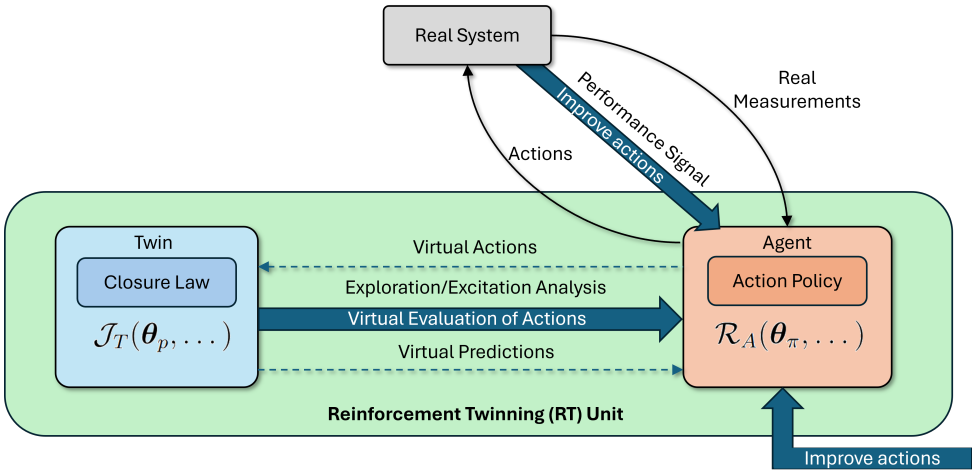


Figure 2: Modeling-to-control regime of reinforcement twinning. The digital twin primarily serves the acting agent: predictive simulations and model updates are exploited to improve control performance. This mode encompasses adaptive control, model predictive control, and model-based reinforcement learning, where models are used to synthesize or optimize control actions (see Section 3.1).

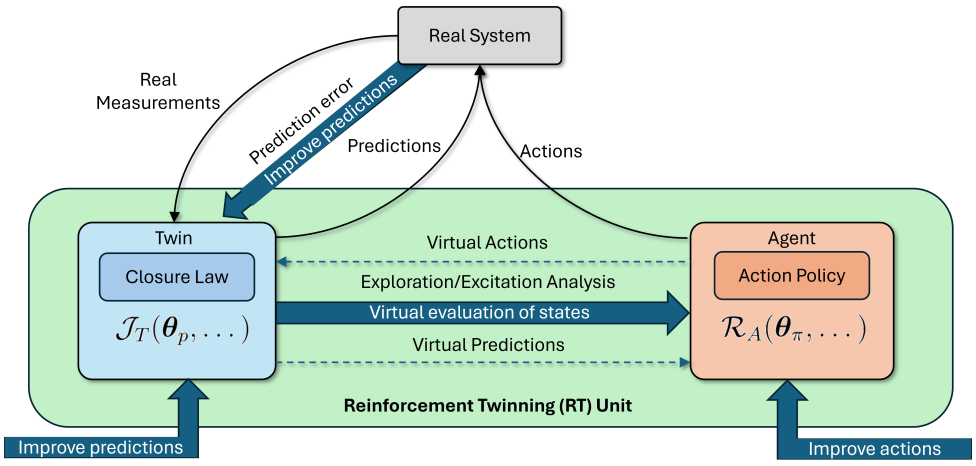


Figure 3: Control-to-modeling regime of reinforcement twinning. Action selection emphasizes informative excitation of the system to improve model fidelity. This mode includes dual control, active identification, and information-theoretic or curiosity-driven exploration approaches (see Section 3.2.

3 Conceptual Roots

The reinforcement twinning architecture lies at the intersection of several established research traditions concerned with the interaction between modeling and control. Historically, these traditions have emphasized one of two complementary directions. In the *modeling-to-control* view, predictive models are built or updated to design or improve control actions. In the *control-to-modeling* view, control actions are chosen to improve system identification, reduce uncertainty, or clarify system structure.

Reinforcement twinning offers a unifying framework in which these two directions

appear as limiting cases of a single cyber-physical learning architecture. Instead of treating modeling and control as sequential or separate, it couples predictive refinement and policy optimization within a persistent feedback loop. The main methods for each direction are reviewed in Sections 3.1 and 3.2.

3.1 Modeling-to-Control

The modeling-to-control paradigm is historically the dominant framework in control theory, and particularly in flow control. In this perspective, a predictive model of the underlying dynamics is constructed and subsequently used to derive an optimal control law. We covered this framework in lecture 5, Section 2.

In fluid mechanics, this tradition spans a broad spectrum of modeling strategies. At one end lie analytical or semi-analytical reductions of the governing equations, enabling optimal control formulations directly at the PDE level (Sritharan, 1998; Gunzburger, 2002). The associated control problems are typically addressed using Pontryagin's Maximum Principle or adjoint-based optimization techniques (Dixon, 1981; Carnarius et al., 2011), or, in principle, through Hamilton–Jacobi–Bellman (HJB) formulations (Peng, 1992; Bardi and Capuzzo-Dolcetta, 1997; Stengel, 1994). These approaches provide a rigorous mathematical foundation, with explicit optimality conditions.

At higher Reynolds numbers or in complex geometries, direct optimal control of the full-order system becomes computationally prohibitive. This has motivated the development of projection-based reduced-order models (ROMs), which approximate the high-dimensional flow dynamics on low-dimensional manifolds extracted from dominant coherent structures (Ravindran, 2000; Rowley and Dawson, 2017). Model reduction enables tractable design of linear-quadratic regulators (LQR) and model predictive controllers (MPC) for fluid flows (see Ghiglieri and Ulbrich (2014); Schwenzer et al. (2021) among others). In these settings, the model acts as a surrogate for the true dynamics, allowing control design to proceed offline with limited direct interaction with the physical system.

The strength of this modeling-to-control family lies in its mathematical structure and sample efficiency. Once a sufficiently accurate model is available, optimal control policies can be synthesized with minimal experimental interaction. However, practical performance depends critically on model fidelity. In real engineering systems, parameters drift, boundary conditions change, and unmodeled phenomena emerge, giving rise to the well-known model–plant mismatch problem (Badwe et al., 2010). The resulting "reality gap" between simulation and operation can degrade performance.

To mitigate these effects, system identification and data assimilation techniques are commonly employed to update model parameters or states using measurement data (Ljung, 1999; Asch et al., 2016). In fluid flows, data-driven closure modeling and machine-learning-enhanced reduced-order models further extend this idea (Chiuso and Pillonetto, 2019; Buizza et al., 2022; Abarbanel et al., 2018). Alternatively, adaptive control strategies adjust controller parameters online to compensate for model uncertainties (Åström and Wittenmark, 2008). In all these cases, the objective remains performance-oriented: model updates are performed to preserve or improve control quality.

More recently, model-based reinforcement learning (MBRL) has revived the modeling-to-control principle within the machine-learning community (Sutton and Barto, 1998; Polydoros and Nalpantidis, 2017; Moerland et al., 2023). Learned dynamics models are

exploited to generate synthetic rollouts, warm-start policies as we saw in tutorial 4 of lecture 5, or guide value-function estimation. In fluid mechanics, reinforcement learning has been successfully applied to active flow control problems (Werner et al., 2023; Pino et al., 2023), often relying on simulators for policy training before deployment. Hybrid approaches further combine model-based structure with model-free flexibility, for instance through residual policy learning or model-guided initialization (Bansal et al., 2017; Nagabandi et al., 2018; Johannink et al., 2019b).

Across this broad spectrum of methodologies—from PDE-constrained optimal control to reduced-order modeling, adaptive control, and model-based reinforcement learning—the unifying principle remains consistent: predictive models are constructed and refined in order to synthesize effective control actions. Reinforcement twinning encompasses this regime when the digital twin primarily serves as a predictive engine supporting policy optimization (Figure 2).

3.2 Control-to-Modeling

In the control-to-modeling paradigm, control actions are chosen to improve system knowledge. Standard adaptive control adopts this idea but largely separates model refinement from control synthesis. This separation leads to the certainty-equivalence principle, where estimated parameters are treated as true when computing control actions (Åström and Wittenmark, 1995; Ioannou and Sun, 1996). As a result, it often ignores that actions stabilizing the system may not reduce model uncertainty. Learning can then remain passive and confined to a narrow range of operating conditions (Feldbaum, 1960), as discussed in Lecture 5, Section 4.

Many approaches acknowledge uncertainty without actively seeking to reducing it. Early examples include central-tendency controllers, which bias adaptive laws toward conservative parameter estimates to mitigate risk (Ryall and Moore, 1989; Moore et al., 1989). More recent uncertainty-aware control and learning methods aim to maximize the probability of meeting performance and safety objectives given explicit uncertainty estimates, e.g., robust or risk-sensitive MPC and uncertainty-aware reinforcement learning (Dehkordi et al., 2025; Kahn et al., 2017). Although effective for safety, these frameworks are mostly exploitative: they manage uncertainty without explicitly choosing actions to gain information.

Another line of work enforces exploration via *persistent excitation*. Classic examples include adaptive pole-positioning (Anderson and Johnstone, 1985) and extremum-seeking control, where dithering signals probe and improve performance (Krstić, 2000; Ariyur and Krstic, 2003). A key limitation is the lack of clear principles for *when* to excite the system and *how* to design probing signals; excitation remains heuristic and most importantly, problem-dependent.

In the case of sloshing dynamics, for example small-amplitude oscillations may suffice for feedback regulation but are generally inadequate for identifying damping or nonlinear free-surface effects. In contrast, deliberate excitation over selected frequency bands improves model fidelity by exposing more of the system's dynamic response. The same applies to variable-pitch propellers: steady operation at an optimal thrust reveals little about aerodynamic coefficients, whereas transient pitch maneuvers produce richer dynamics that facilitate parameter estimation. Likewise, running a wind turbine only near its

optimal tip-speed ratio yields nor information about off-design aerodynamics (see tutorials 2 and 3 of lecture 5), while deliberate excursions—such as brief operation at high induction factors—reveal nonlinear load responses that enable more accurate models. Similarly, tightly regulated cryogenic storage units, though vital for safety and nominal performance, can mask heat-transfer nonlinearities that appear only during controlled thermal transients or dynamic excitations such as sloshing.

A more principled framework linking regulation and learning is *dual control*, which chooses control actions to jointly optimize performance and improve state and/or parameter estimates (Feldbâum, 1963; Wittenmark, 1995; Mesbah, 2018). In principle, dynamic programming addresses this trade-off by valuing actions that reduce uncertainty when this lowers expected long-term cost (Bertsekas, 2005; Chen et al., 2021). Probing becomes optimal whenever its expected future benefit exceeds its short-term performance loss. However, exact dynamic programming is computationally intractable for most coupled estimation–control problems (Åström and Helmersson, 1986; Bernhardsson, 1989).

Practical methods approximate the exploration–exploitation trade-off with surrogate mechanisms. A common approach is to augment the control objective with explicit estimation-quality terms, coupling regulation and identification in one criterion (Goodwin and Payne, 1977; Wittenmark and Elevitch, 1985). In many applications, the balance between performance and information gathering is enforced indirectly via surrogate costs, separate identification phases, or heuristic excitation.

Beyond dual control, related fields also exemplify the control-to-modeling principle. In adaptive system identification and optimal experiment design, input trajectories are optimized to maximize information or parameter identifiability (Goodwin and Payne, 1977; Ljung, 1999), with control signals chosen to improve the model rather than regulate. In reinforcement learning, curiosity-driven and intrinsic-motivation methods reward actions that reduce prediction error or increase model uncertainty, formalizing information-seeking in high-dimensional spaces (Schmidhuber, 2010; Pathak et al., 2017). Though often framed within AI, these approaches share a core idea: control shapes the data distribution to improve predictions.

Model-based fault detection and diagnosis provide a related view (Isermann, 2006; Blanke et al., 2016). Comparing model predictions with measurements enables residual-based fault detection, and in active fault diagnosis, control inputs are designed to increase diagnostic sensitivity. The goal shifts from regulation to structural discrimination. In sloshing tanks, specific excitation patterns can amplify residuals indicating structural changes such as altered boundary conditions or internal damping. In propulsion systems, deliberate pitch modulation in a VPP can increase sensitivity to blade asymmetries or actuator degradation. Here, control actions enhance diagnostic observability.

Across these traditions, the idea is consistent: actions influence knowledge. The control-to-modeling regime of reinforcement twinning generalizes this by embedding information seeking behavior.

4 General Mathematical Formulation

Reinforcement twinning can be formalized in discrete time as a coupled cyber–physical dynamical system composed of three layers: the real system, the digital twin, and a

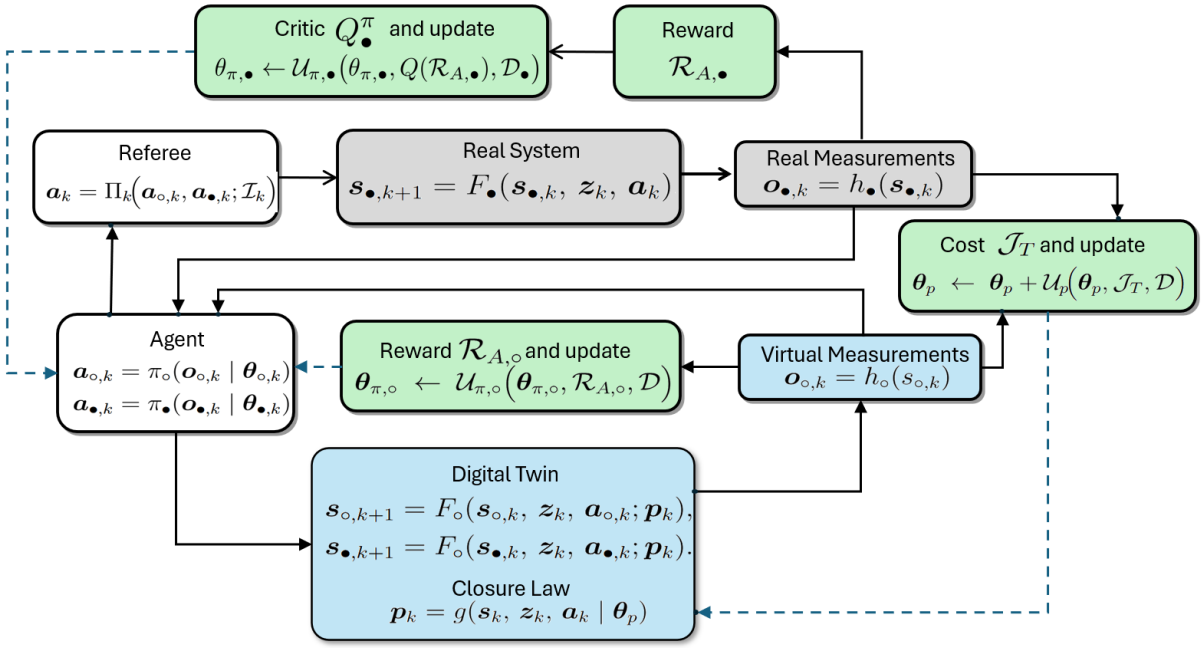


Figure 4: Reinforcement twinning architecture coupling a digital twin and two candidate policies around a real system. Prediction error updates the twin via $\mathcal{J}_T$, while rewards $\mathcal{R}_{A,\circ}$ and $\mathcal{R}_{A,\bullet}$ update the policies.

set of candidate policies evaluated within the twin. Figure 4 summarizes this structure schematically, highlighting the coexistence of a physical trajectory and multiple virtual rollouts within the twin, together with the referee mechanism that selects which action is applied to the plant. The extension of this formulation to continuous time is natural, replacing the discrete-time updates of the plant, twin, and candidate policies with the corresponding flows generated by their continuous-time dynamics.

4.1 Structural Components

Real System (Black-Box Layer). The physical system evolves according to unknown dynamics,

$$\begin{cases} \boldsymbol{s}_{\bullet,k+1} = F_{\bullet}(\boldsymbol{s}_{\bullet,k},\ \boldsymbol{z}_k,\ \boldsymbol{a}_k), \\ \boldsymbol{o}_{\bullet,k} = h_{\bullet}(\boldsymbol{s}_{\bullet,k}), \end{cases} \tag{1}$$

where $\boldsymbol{s}_{\bullet,k} \in \mathbb{R}^{n_s}$ denotes the true and is usually not available, $\boldsymbol{z}_k \in \mathbb{R}^{n_z}$ denotes exogenous inputs or disturbances and $\mathbf{a}_k \in \mathbb{R}^{n_a}$ denotes the control action. We return to this later. The policy governing these actions and the training of the digital twin can only rely on partial, indirect and usually noisy observations $\boldsymbol{o}_{\bullet,k} \in \mathbb{R}^{n_o}$, which are linked to the actual state via an unknown observation function $h_{\bullet}()$.

Digital Twin (White-Box Layer). The digital twin provides a structured predictive model in which the dominant physics are represented explicitly, while unresolved or unmodeled effects are captured through a data-driven closure law. In the notation of Lecture 5, we introduce a closure variable $\boldsymbol{p}_k$ generated by a parametric closure model

$$\boldsymbol{p}_k = g(\boldsymbol{s}_{\circ,k},\ \boldsymbol{z}_k,\ \boldsymbol{a}_k \mid \boldsymbol{\theta}_p), \tag{2}$$

where $\boldsymbol{\theta}_p$ denotes the closure parameters to be learned and $\boldsymbol{s}_{\circ,k}$ denotes the full state of the system. This could be inferred from the partial observation using techniques from data assimilation or could be provided by the digital twin, which evolves it according to a fully known dynamics

$$\begin{cases} \boldsymbol{s}_{\circ,k+1} = F_\circ(\boldsymbol{s}_{\circ,k},\, \boldsymbol{z}_k,\, \boldsymbol{a}_k; \boldsymbol{p}_k), \\ \boldsymbol{o}_{\circ,k} = h_\circ(\boldsymbol{s}_{\circ,k}), \end{cases} \tag{3}$$

These "twin" states could also be corrected using the real observations $\boldsymbol{o}_{\bullet,k}$ using standard statistical tools such as Kalman filters. The virtual observations $\boldsymbol{o}_{\circ,k} = h_\circ(s_{\circ,k})$ are designed to mimic the measurement process from the real world and one could expect $h_\circ \neq h_\bullet$ in general.

Remark on observations. In the notation adopted throughout this chapter, the observation vector $\boldsymbol{o}_{\star,k}$ (with $\star \in \{\bullet, \circ\}$) generically denotes the collection of measurable signals available at time step k. This vector may include quantities used for feedback in the policy as well as signals entering the evaluation of the reward. In practice, these two roles need not coincide: some components of $\boldsymbol{o}_{\star,k}$ may be used exclusively for actuation, others exclusively for performance assessment, and others for both. For notational compactness, however, we group them under the single symbol $\boldsymbol{o}_{\star,k}$, understanding that it represents the full information set available to the corresponding learning branch.

Candidate Control Policies. Two candidate policies are considered: one relying from interaction with the real system and another leveraging the digital twin. These are defined as follows

$$\boldsymbol{a}_{\circ,k} = \pi_\circ(\boldsymbol{o}_{\circ,k} \mid \boldsymbol{\theta}_{\pi,\circ}), \tag{4}$$

$$\boldsymbol{a}_{\bullet,k} = \pi_\bullet(\boldsymbol{o}_{\bullet,k} \mid \boldsymbol{\theta}_{\pi,\bullet}), \tag{5}$$

In general, the two policies need not coincide. For instance, one could choose $\pi_\circ$ to be a structured model-based controller (e.g., LQR, MPC), while letting $\pi_\bullet$ denote an artificial neural network trained via reinforcement learning.

Parallel Twin Rollouts. The twin might be used to evaluate both the model-free and the model-based policies. This means that at each time step this can be evolved under both candidate actions, hence giving rise to two virtual trajectories:

$$\boldsymbol{s}_{\circ\circ,k+1} = F_\circ(\boldsymbol{s}_{\circ\circ,k},\, \boldsymbol{z}_k,\, \boldsymbol{a}_{\circ\circ,k} = \pi_\circ(h_\circ(\boldsymbol{s}_{\circ\circ,k}) \mid \boldsymbol{\theta}_{\pi,\circ}) \mid \boldsymbol{p}_k), \tag{6}$$

$$\boldsymbol{s}_{\circ\bullet,k+1} = F_\circ(\boldsymbol{s}_{\circ\bullet,k},\, \boldsymbol{z}_k,\, \boldsymbol{a}_{\circ\bullet,k} = \pi_\bullet(h_\circ(\boldsymbol{s}_{\circ\bullet,k}) \mid \boldsymbol{\theta}_{\pi,\bullet}) \mid \boldsymbol{p}_k)\,. \tag{7}$$

Computing for each of these the reward $\mathcal{R}_{A,\circ}$ allows to evaluate the quality of the two policies according to the digital twin; we return to the reward definition in the following section. It is worth stressing that only one of the two actions is actually applied to the real system (1) according to the referee mechanism described below. Therefore, only the rollout associated with the action applied to the real world can be expected to be aligned with the physical evolution and thus used to compute tracking errors and the digital twin performance.

Referee Mechanism (Live vs. Idle Policy). At the beginning of each episode, the referee identifies the policy that will interact with the real system and thus define the

actions in (1). Many options are possible and deserves detailed investigation. Similarly to Schena et al. (2024); Poletti et al. (2025), the approach presented in these notes considers

$$\boldsymbol{a}_k = \Pi_{(e)}\Big(\pi_\bullet(\boldsymbol{o}_{\bullet,k} \mid \boldsymbol{\theta}_{\pi,\bullet}), \pi_\circ(\boldsymbol{o}_{\bullet,k} \mid \boldsymbol{\theta}_{\pi,\circ}) \mid \mathcal{I}_{(e)}\Big), \tag{8}$$

where $\mathcal{I}_{(e)}$ denotes supervisory information such as safety constraints, confidence measures, or predicted performance. In this formulation the selected policy is applied to real observations and remains active for the full episode. Alternative strategies may allow step-wise switching or arbitration based on virtual states or state-estimation outputs. The non-selected policy remains idle with respect to the plant but continues to be evaluated within the twin. The rationale behind maintaining two policies in parallel is that the idle policy, although not interacting directly with the real system, continues to improve through twin rollouts and indirect information. When its predicted or observed performance exceeds that of the live policy, the referee can promote it to govern the plant. This virtuous loop, in which the learner may eventually surpass the master, was observed in Schena et al. (2024).

Decoupled and Coupled Learning Mechanisms

The current reinforcement twinning framework is formulated in an episodic setting. Over an episode involving n_t iterations (assumed for simplicity to be equally spaced in time), the real system generates a sequence of observations $\{\boldsymbol{o}_{\bullet,k}\}_{k=1}^{n_t}$ under the applied action sequence $\{\boldsymbol{a}_k\}_{k=1}^{n_t}$ selected by the referee according to the unknown dynamics (1) and driven by a realized disturbance sequence $\{\boldsymbol{z}_k\}_{k=1}^{n_t}$. In parallel, the digital twin generates the two virtual trajectories (6)–(7).

Twin-only update. The identification cost for a single episode is defined as

$$\mathcal{J}_T^{(e)}(\boldsymbol{\theta}_p \mid \boldsymbol{\theta}_{\pi,\circ}, \boldsymbol{\theta}_{\pi,\bullet}) = \sum_{k=0}^{n_t} \ell_{\text{pred}}\Big(\boldsymbol{o}_{\bullet,k}, \boldsymbol{o}_{\circ,k}(\boldsymbol{\theta}_p)\Big), \tag{9}$$

where $\boldsymbol{o}_{\circ,k} = h_\circ(\boldsymbol{s}_{\circ\star,k})$ and $\ell_{\text{pred}}(\cdot)$ measures the prediction error. If the disturbance sequence is not directly measurable, a disturbance model must be introduced (e.g. via estimation or data assimilation), and the expected identification objective over a disturbance distribution $\boldsymbol{z} \sim \mathcal{Z}$ may be considered.

Policy Reward functionals. At the same time, the acting policies are driven by accumulated rewards reflecting control performance. Denoting by $r(\boldsymbol{s}, \boldsymbol{a})$ the stage reward encoding regulation quality, tracking accuracy, energy efficiency, constraint satisfaction, or economic performance, the cumulative reward driving the model-free policy search is

$$\mathcal{R}_{A,\bullet}^{(e)}\Big(\boldsymbol{\theta}_{\pi,\bullet} \mid \boldsymbol{\theta}_{\pi,\circ}\Big) = \sum_{k=0}^{n_t-1} r\Big(\boldsymbol{s}_{\bullet,k}^{(e)}, \pi_\bullet(\boldsymbol{o}_{\bullet,k}^{(e)} \mid \boldsymbol{\theta}_{\pi,\bullet})\Big), \tag{10}$$

The corresponding transition memory buffer collected from the real system is

$$\mathcal{D}_\bullet = \Big\{(\boldsymbol{o}_{\bullet,k}, \boldsymbol{a}_k, r_k, \boldsymbol{o}_{\bullet,k+1})\Big\}_{k=1}^{N_{\text{tot}}}, \tag{11}$$

where N_{tot} denotes the total number of collected transitions across episodes on the real system. Importantly, this buffer aggregates data regardless of whether actions were generated by the model-free or model-based policy, thereby creating an intrinsic interaction channel between the two branches.

Similarly, the cumulative reward driving the model-based policy search is computed from the white rollout in (6), namely

$$\mathcal{R}_{A,\circ}^{(e)}\Big(\boldsymbol{\theta}_{\pi,\circ} \mid \boldsymbol{\theta}_p\Big) = \sum_{k=0}^{n_t-1} r\Big(\boldsymbol{s}_{\circ\circ,k}^{(e)}(\boldsymbol{\theta}_p), \pi_\circ(\boldsymbol{o}_{\circ,k}^{(e)} \mid \boldsymbol{\theta}_{\pi,\circ})\Big)\,. \tag{12}$$

4.2 Decoupled and Coupled Learning Mechanisms

Before discussing their interaction, it is instructive to consider the update mechanisms in isolation. All the isolated mechanisms were covered in detail in lecture 5, to which the reader is referred for more information.

Twin-only update. Across episodes, identification data accumulate as

$$\mathcal{D}_T = \bigcup_{e=1}^{N_e} \mathcal{D}_T^{(e)}, \qquad \mathcal{D}_T^{(e)} = \{\boldsymbol{o}_{\bullet,k}, \boldsymbol{a}_k, \boldsymbol{z}_k\}_{k=1}^{n_t}.$$

Twin parameters evolve according to

$$\boldsymbol{\theta}_p^{(e+1)} = \mathcal{U}_p\Big(\boldsymbol{\theta}_p^{(e)}, \mathcal{J}_T^{(e)}, \mathcal{D}_T\Big), \tag{13}$$

where $\mathcal{U}_p$ denotes an abstract update operator (e.g. stochastic gradient descent, Bayesian posterior update, acquisition-driven sampling, or metaheuristic search).

Model-free-only update. The model-free parameters evolve as

$$\boldsymbol{\theta}_{\pi,\bullet}^{(e+1)} = \mathcal{U}_{\pi,\bullet}\Big(\boldsymbol{\theta}_{\pi,\bullet}^{(e)}, \mathcal{R}_{A,\bullet}^{(e)}, \mathcal{D}_\bullet\Big), \tag{14}$$

where $\mathcal{U}_{\pi,\bullet}$ is the corresponding update operator. The buffer $\mathcal{D}_\bullet$ is persistent and may be sampled stochastically to improve numerical stability and mitigate catastrophic forgetting.

Model-based-only update. For the model-based branch, a batch of simulated trajectories is generated within the twin at each episode,

$$\mathcal{B}_\circ^{(e)} = \Big\{(\boldsymbol{o}_{\circ,k}, \boldsymbol{a}_{\circ,k}, r_{\circ,k}, \boldsymbol{o}_{\circ,k+1})\Big\}_{k=1}^{N_\circ}, \tag{15}$$

with $r_{\circ,k} = r(\boldsymbol{s}_{\circ\circ,k}, \boldsymbol{a}_{\circ,k})$.

Policy parameters evolve according to

$$\boldsymbol{\theta}_{\pi,\circ}^{(e+1)} = \mathcal{U}_{\pi,\circ}\Big(\boldsymbol{\theta}_{\pi,\circ}^{(e)}, \mathcal{R}_{A,\circ}^{(e)}, \mathcal{B}_\circ^{(e)}\Big). \tag{16}$$

where $\mathcal{B}_\circ^{(e)}$ is generated on demand and need not be stored persistently.

Coupled Reinforcement Twinning update. Reinforcement Twinning arises when these mechanisms are no longer independent. The identification dataset $\mathcal{D}_T^{(e)}$ depends on the active policy since control actions shape the state–action distribution from which data are collected. Conversely, the model-based update depends explicitly on the current twin parameters, and the model-free branch may exploit twin information through demonstrations, reward shaping, or weighted virtual transitions.

The coupled updates may therefore be written abstractly as

$$\boldsymbol{\theta}_p^{(e+1)} = \mathcal{U}_p\Big(\boldsymbol{\theta}_p^{(e)}, \mathcal{D}_T^{(e)}(\boldsymbol{\theta}_{\pi,\bullet}^{(e)}, \boldsymbol{\theta}_{\pi,\circ}^{(e)})\Big), \tag{17}$$

$$\boldsymbol{\theta}_{\pi,\bullet}^{(e+1)} = \mathcal{U}_{\pi,\bullet}\Big(\boldsymbol{\theta}_{\pi,\bullet}^{(e)}, \mathcal{D}_\bullet, \boldsymbol{\theta}_p^{(e)}\Big), \tag{18}$$

$$\boldsymbol{\theta}_{\pi,\circ}^{(e+1)} = \mathcal{U}_{\pi,\circ}\Big(\boldsymbol{\theta}_{\pi,\circ}^{(e)}, \mathcal{B}_\circ^{(e)}(\boldsymbol{\theta}_p^{(e)}), \boldsymbol{\theta}_{\pi,\bullet}^{(e)}\Big). \tag{19}$$

Reinforcement Twinning is thus characterized by the co-evolution of model and policy parameters within a shared feedback architecture rather than by any specific optimization algorithm.

4.3 Scope and Generality of the Framework

Reinforcement Twinning is formulated as a general architectural framework rather than a specific algorithm. The update operators $\mathcal{U}_p$, $\mathcal{U}_{\pi,\bullet}$, and $\mathcal{U}_{\pi,\circ}$ are intentionally left abstract, allowing gradient-based optimization, Bayesian inference, evolutionary search, or surrogate-assisted strategies to be embedded within the same structural loop.

In this chapter, we focus primarily on the modeling-to-control regime, in which improvements in $\mathcal{R}_{A,\star}$ are guided by the predictive structure of the twin and interaction with the real system, while $\mathcal{J}_T$ ensures consistency with measured data. The reverse control-to-modeling regime—where actions are selected to accelerate the reduction of $\mathcal{J}_T$ through informative excitation—is only briefly outlined and left for future elaboration. We restrict attention to a single reinforcement twinning unit and defer collective and distributed feedback mechanisms to later work.

We consider in the following a formulation that blends adjoint-based optimal control with actor–critic learning, illustrating gradient-level interaction between model-based and model-free updates.

5 Optimal Control and Actor–Critic Learning

5.1 The actor–critic search in TD3

The formulation presented in this section extends the actor–critic search introduced in Schena et al. (2024) and refined in Poletti et al. (2025). In the present work, the model-free policy is trained using the Twin Delayed Deep Deterministic Policy Gradient (TD3) algorithm (Fujimoto et al., 2018), augmented with Prioritized Experience Replay (PER) (Schaul et al., 2015) and a behavioral cloning (BC, Torabi et al. (2018)) regularization term based on expert trajectories generated by the model-based policy.

As in the actor–critic reinforcement learning formalism (Grondman et al., 2012), the algorithm trains both a critic (i.e. a model approximating the state–action value function Q) and an actor (that is, a policy) mapping states to actions. Both are typically parametrized using artificial neural networks.

As introduced in Lecture 5, a stage reward $r(\boldsymbol{s}, \boldsymbol{a})$ induces an action–value function (or Q-function) defined, in the episodic setting, as the expected cumulative return obtained by applying action $\boldsymbol{a}$ in state $\boldsymbol{s}$ and subsequently following a given policy.

In the real system, this may be written as

$$Q^{\pi}_{\bullet}(\boldsymbol{o}, \boldsymbol{a}) = \mathbb{E}\left[\sum_{k=0}^{n_t-1} r(\boldsymbol{s}_{\bullet,k}, \boldsymbol{a}_k) \;\middle|\; \boldsymbol{o}_{\bullet,0} = \boldsymbol{o},\ \boldsymbol{a}_0 = \boldsymbol{a},\ \pi \right], \tag{20}$$

where the expectation is taken over the stochastic evolution of the real system and disturbances. Estimates of this function drive the training of the model-free actor.

Let the model-free policy be parametrized as

$$\boldsymbol{a}_{\bullet,k} = \pi_{\bullet}(\boldsymbol{o}_{\bullet,k} \mid \boldsymbol{\theta}_{\pi,\bullet}), \tag{21}$$

with $\boldsymbol{\theta}_{\pi,\bullet}$ the policy parameters (typically weights and biases of a neural network). In parallel, the model-based policy used to generate expert trajectories is defined as

$$\boldsymbol{a}_{\circ,k} = \pi_{\circ}(\boldsymbol{o}_{\circ,k} \mid \boldsymbol{\theta}_{\pi,\circ}). \tag{22}$$

In TD3, two critics are trained in parallel to approximate the real action–value function,

$$Q^{\pi}_{\bullet}(\boldsymbol{o}, \boldsymbol{a}) \approx Q^{\pi}_{\bullet,j}(\boldsymbol{o}, \boldsymbol{a}; \boldsymbol{\theta}_{\bullet Q_j}), \qquad j \in \{1, 2\}, \tag{23}$$

where $\boldsymbol{\theta}_{\bullet Q_1}$ and $\boldsymbol{\theta}_{\bullet Q_2}$ denote the critic parameters. The use of twin critics mitigates the positive bias that may arise when a single critic is used to construct bootstrapped targets.

Following the deterministic policy gradient framework (Silver et al., 2014), the gradient of the expected cumulative reward with respect to the actor parameters, evaluated at the current iterate $\boldsymbol{\theta}^{(e)}_{\pi,\bullet}$, can be approximated using a mini-batch of N transitions sampled from the replay buffer $\mathcal{D}_{\bullet}$ as

$$\nabla_{\boldsymbol{\theta}_{\pi,\bullet}} \mathcal{R}_{A,\bullet}\Big|_{\boldsymbol{\theta}^{(e)}_{\pi,\bullet}} \approx \frac{1}{N} \sum_{i=1}^{N} \nabla_{\boldsymbol{\theta}_{\pi,\bullet}} \pi_{\bullet}(\boldsymbol{o}^{(i)}_{\bullet} \mid \boldsymbol{\theta}^{(e)}_{\pi,\bullet}) \nabla_{\boldsymbol{a}} Q^{\pi}_{\bullet,1}(\boldsymbol{o}^{(i)}_{\bullet}, \boldsymbol{a}; \boldsymbol{\theta}_{\bullet Q_1})\Big|_{\boldsymbol{a}=\pi_{\bullet}(\boldsymbol{o}^{(i)}_{\bullet} \mid \boldsymbol{\theta}^{(e)}_{\pi,\bullet})}. \tag{24}$$

In practice, TD3 uses one of the two critics (here $Q^{\pi}_{\bullet,1}$) for the actor update, while both critics are used to construct stabilized Bellman targets.

In the present formulation, the actor update is regularized by a behavioral cloning term on expert state–action pairs collected from the model-based policy. Let $\mathcal{D}^{\mathrm{exp}}_{\circ}$ denote the expert dataset generated by $\pi_{\circ}$. Then, at actor update step e, the model-free actor is trained by minimizing the augmented objective

$$\mathcal{L}_{A,\bullet}\big(\boldsymbol{\theta}_{\pi,\bullet}\big) = -\frac{1}{N} \sum_{i=1}^{N} Q^{\pi}_{\bullet,1}\Big(\boldsymbol{o}^{(i)}_{\bullet}, \pi_{\bullet}(\boldsymbol{o}^{(i)}_{\bullet} \mid \boldsymbol{\theta}_{\pi,\bullet}); \boldsymbol{\theta}_{\bullet Q_1}\Big) + \lambda_{\mathrm{BC}} \frac{1}{N_{\mathrm{exp}}} \sum_{j=1}^{N_{\mathrm{exp}}} \Big\| \pi_{\bullet}(\boldsymbol{o}^{(j)}_{\circ} \mid \boldsymbol{\theta}_{\pi,\bullet}) - \boldsymbol{a}^{(j)}_{\circ} \Big\|_2^2, \tag{25}$$

where $\lambda_{\mathrm{BC}} \geq 0$ controls the strength of imitation. The first term promotes actions with high predicted return under the real critic, while the second term encourages the actor to remain close to the expert policy on the expert trajectories.

The corresponding gradient of the augmented actor objective is therefore

$$\nabla_{\boldsymbol{\theta}_{\pi,\bullet}} \mathcal{L}_{A,\bullet} = -\nabla_{\boldsymbol{\theta}_{\pi,\bullet}} \mathcal{R}_{A,\bullet} + \lambda_{\mathrm{BC}} \nabla_{\boldsymbol{\theta}_{\pi,\bullet}} \left[\frac{1}{N_{\mathrm{exp}}} \sum_{j=1}^{N_{\mathrm{exp}}} \left\| \pi_\bullet(\boldsymbol{o}_\circ^{(j)} \mid \boldsymbol{\theta}_{\pi,\bullet}) - \boldsymbol{a}_\circ^{(j)} \right\|_2^2 \right], \tag{26}$$

with $\nabla_{\boldsymbol{\theta}_{\pi,\bullet}} \mathcal{R}_{A,\bullet}$ approximated by (24). This decomposition is useful later when comparing actor updates in the model-free and model-based settings.

Although the gradient in (24) (and thus the first term in (26)) is evaluated at the current parameter value $\boldsymbol{\theta}_{\pi,\bullet}^{(e)}$, the sampled transitions stored in the replay buffer $\mathcal{D}_\bullet$ were generally generated by earlier versions of the policy. The distribution of visited states does not coincide with the distribution induced by the current actor. Because the update relies on data collected under a behavior policy different from the current policy being optimized, the actor update is performed *off-policy*. The replay buffer thus decouples data collection from policy improvement, enabling more stable learning and more efficient reuse of past experience.

In parallel with the actor update, both critics are trained by minimizing Bellman residuals on sampled transitions. For each critic $j \in \{1, 2\}$,

$$\mathcal{L}_{\bullet Q_j}(\boldsymbol{\theta}_{\bullet Q_j}) = \frac{1}{N} \sum_{i=1}^{N} w^{(i)} \left(Q_{\bullet,j}^{\pi}(\boldsymbol{o}_\bullet^{(i)}, \boldsymbol{a}^{(i)}; \boldsymbol{\theta}_{\bullet Q_j}) - y^{(i)} \right)^2, \tag{27}$$

where $w^{(i)}$ are the importance-sampling weights introduced by PER, and the target is defined using target networks and clipped double-Q learning as

$$y^{(i)} = r^{(i)} + \gamma \min_{\ell \in \{1,2\}} Q_{\bullet,\ell}^{\pi}\left(\boldsymbol{o}_\bullet'^{(i)}, \tilde{\boldsymbol{a}}^{(i)}; \boldsymbol{\theta}_{\bullet Q_\ell}^{\mathrm{targ}} \right), \tag{28}$$

with target action

$$\tilde{\boldsymbol{a}}^{(i)} = \pi_\bullet(\boldsymbol{o}_\bullet'^{(i)} \mid \boldsymbol{\theta}_{\pi,\bullet}^{\mathrm{targ}}) + \boldsymbol{\epsilon}^{(i)}, \qquad \boldsymbol{\epsilon}^{(i)} \sim \mathrm{clip}\left(\mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}), -\bar{\epsilon}, \bar{\epsilon} \right), \tag{29}$$

where clipped target noise is used for target-policy smoothing, as in TD3 (Fujimoto et al., 2018). If the action space is bounded, $\tilde{\boldsymbol{a}}^{(i)}$ is additionally projected onto the admissible action set before evaluation of the target critic.

Here $\boldsymbol{\theta}_{\pi,\bullet}^{\mathrm{targ}}$ and $\boldsymbol{\theta}_{\bullet Q_j}^{\mathrm{targ}}$ denote target-network parameters that evolve slowly according to

$$\boldsymbol{\theta}^{\mathrm{targ}} \leftarrow \tau\, \boldsymbol{\theta} + (1 - \tau)\, \boldsymbol{\theta}^{\mathrm{targ}}, \qquad 0 < \tau \ll 1, \tag{30}$$

which improves numerical stability by reducing oscillations in the Bellman targets. In addition, TD3 updates the actor less frequently than the critics (delayed policy updates), which further stabilizes training by allowing the critics to track the Bellman targets more accurately before each policy improvement step.

In prioritized replay (Schaul et al., 2015), the temporal-difference error in (27) ranks each transition in the replay buffer, giving higher sampling probability to those with larger Bellman residuals. Transitions with larger prediction errors are treated as more informative for improving the critics and are revisited more often. To prevent bias, importance-sampling weights $w^{(i)}$ are included in (27) so that the resulting stochastic gradients remain consistent with the underlying objective.

5.2 From Q-Functions to Hamiltonian

The gradient in (24) deserves a long pause. The reader is encouraged to move back to the notes in lecture 5 and look for a parallelism in the update rule in the model based formalism, which is here carried out in the virtual environment. Having knowledge of the system dynamics $\boldsymbol{s}_{\circ,k+1} = F_\circ(\boldsymbol{s}_{\circ,k}, \boldsymbol{z}_k, \boldsymbol{a}_{\circ,k}; \boldsymbol{\theta}_p)$, allows to differentiate the virtual reward and lead to the expression

$$\nabla_{\boldsymbol{\theta}_{\pi,\circ}} \mathcal{R}_{A,\circ} = \sum_{k=0}^{n_t-1} \frac{\partial \mathcal{H}_k}{\partial \boldsymbol{a}_{\circ,k}} \frac{\partial \pi_\circ(\boldsymbol{o}_{\circ,k})}{\partial \boldsymbol{\theta}_{\pi,\circ}} = \sum_{k=0}^{n_t-1} \Big(\nabla_{\boldsymbol{\theta}_{\pi,\circ}} \pi_\circ(\boldsymbol{o}_{\circ,k})\Big)^\top \nabla_{\boldsymbol{a}_{\circ,k}} \mathcal{H}_k \Big|_{\boldsymbol{a}_{\circ,k}=\pi_\circ(\boldsymbol{o}_{\circ,k})}. \tag{31}$$

where the discrete Hamiltonian is defined as

$$\mathcal{H}_k = r(\boldsymbol{s}_{\circ,k}, \boldsymbol{a}_{\circ,k}) + \boldsymbol{\lambda}_{\circ,k+1}^\top F_\circ(\boldsymbol{s}_{\circ,k}, \boldsymbol{z}_k, \boldsymbol{a}_{\circ,k}; \boldsymbol{\theta}_p), \tag{32}$$

and the adjoint variables $\boldsymbol{\lambda}_{\circ,k}$ evolve backward in time according to

$$\boldsymbol{\lambda}_{\circ,k} = \frac{\partial r}{\partial \boldsymbol{s}_{\circ,k}} + \left(\frac{\partial F_\circ}{\partial \boldsymbol{s}_{\circ,k}}\right)^\top \boldsymbol{\lambda}_{\circ,k+1}, \qquad \boldsymbol{\lambda}_{\circ,n_t} = 0, \tag{33}$$

as detailed in Lecture 5.

Comparing (31) with (24) is particularly illuminating: it appears that the gradient of the Hamiltonian with respect to the action is playing the same role as the gradient of the Q function with respect to the action. This is worth digging.

Let us define the (finite-horizon) cost-to-go (value function) along the twin dynamics,

$$V_{\circ,k}(\boldsymbol{s}_{\circ,k}) := \sum_{t=k}^{n_t-1} r(\boldsymbol{s}_{\circ,t}, \boldsymbol{a}_{\circ,t}), \qquad \boldsymbol{s}_{\circ,t+1} = F_\circ(\boldsymbol{s}_{\circ,t}, \boldsymbol{z}_t, \boldsymbol{a}_{\circ,t}; \boldsymbol{\theta}_p), \tag{34}$$

where the action sequence $\{\boldsymbol{a}_{\circ,t}\}_{t=k}^{n_t-1}$ is generated by a given policy (or by an optimal decision rule) in the virtual environment. By construction, $V_{\circ,k}$ satisfies the one-step recursion

$$V_{\circ,k}(\boldsymbol{s}_{\circ,k}) = r(\boldsymbol{s}_{\circ,k}, \boldsymbol{a}_{\circ,k}) + V_{\circ,k+1}(\boldsymbol{s}_{\circ,k+1}), \tag{35}$$

Switching back to the Jacobian notation of Lecture 5 and using the chain rule, differentiating (35) with respect to the current state gives

$$\frac{\partial V_{\circ,k}}{\partial \boldsymbol{s}_{\circ,k}} = \frac{\partial r(\boldsymbol{s}_{\circ,k}, \boldsymbol{a}_{\circ,k})}{\partial \boldsymbol{s}_{\circ,k}} + \frac{\partial V_{\circ,k+1}}{\partial \boldsymbol{s}_{\circ,k+1}} \frac{\partial \boldsymbol{s}_{\circ,k+1}}{\partial \boldsymbol{s}_{\circ,k}}, \tag{36}$$

and, using $\boldsymbol{s}_{\circ,k+1} = F_\circ(\boldsymbol{s}_{\circ,k}, \boldsymbol{z}_k, \boldsymbol{a}_{\circ,k}; \boldsymbol{\theta}_p)$,

$$\frac{\partial V_{\circ,k}}{\partial \boldsymbol{s}_{\circ,k}} = \frac{\partial r(\boldsymbol{s}_{\circ,k}, \boldsymbol{a}_{\circ,k})}{\partial \boldsymbol{s}_{\circ,k}} + \frac{\partial V_{\circ,k+1}}{\partial \boldsymbol{s}_{\circ,k+1}} \frac{\partial F_\circ(\boldsymbol{s}_{\circ,k}, \boldsymbol{z}_k, \boldsymbol{a}_{\circ,k}; \boldsymbol{\theta}_p)}{\partial \boldsymbol{s}_{\circ,k}}. \tag{37}$$

This expression should be contrasted with (33) and looked at long enough to realize that the (row) costate gives

$$\boldsymbol{\lambda}_{\circ,k}^\top := \frac{\partial V_{\circ,k}}{\partial \boldsymbol{s}_{\circ,k}}, \qquad \boldsymbol{\lambda}_{\circ,k+1}^\top := \frac{\partial V_{\circ,k+1}}{\partial \boldsymbol{s}_{\circ,k+1}}, \tag{38}$$

so that (37) can be rewritten as the backward propagation of costate sensitivities.

This observation becomes even more revealing when combined with the relationship between the value function and the state–action value function:

$$Q_{\circ,k}(\boldsymbol{s}_{\circ,k}, \boldsymbol{a}_{\circ,k}) = r(\boldsymbol{s}_{\circ,k}, \boldsymbol{a}_{\circ,k}) + V_{\circ,k+1}(F_\circ(\boldsymbol{s}_{\circ,k}, \boldsymbol{z}_k, \boldsymbol{a}_{\circ,k}; \boldsymbol{\theta}_p)) \,. \tag{39}$$

Differentiating (39) with respect to the current action and applying the chain rule gives

$$\frac{\partial Q_{\circ,k}}{\partial \boldsymbol{a}_{\circ,k}} = \frac{\partial r(\boldsymbol{s}_{\circ,k}, \boldsymbol{a}_{\circ,k})}{\partial \boldsymbol{a}_{\circ,k}} + \frac{\partial V_{\circ,k+1}}{\partial \boldsymbol{s}_{\circ,k+1}} \frac{\partial \boldsymbol{s}_{\circ,k+1}}{\partial \boldsymbol{a}_{\circ,k}}, \tag{40}$$

and using $\boldsymbol{s}_{\circ,k+1} = F_\circ(\boldsymbol{s}_{\circ,k}, \boldsymbol{z}_k, \boldsymbol{a}_{\circ,k}; \boldsymbol{\theta}_p)$,

$$\frac{\partial Q_{\circ,k}}{\partial \boldsymbol{a}_{\circ,k}} = \frac{\partial r(\boldsymbol{s}_{\circ,k}, \boldsymbol{a}_{\circ,k})}{\partial \boldsymbol{a}_{\circ,k}} + \frac{\partial V_{\circ,k+1}}{\partial \boldsymbol{s}_{\circ,k+1}} \frac{\partial F_\circ(\boldsymbol{s}_{\circ,k}, \boldsymbol{z}_k, \boldsymbol{a}_{\circ,k}; \boldsymbol{\theta}_p)}{\partial \boldsymbol{a}_{\circ,k}} \tag{41}$$

that, using (38):

$$\frac{\partial Q_{\circ,k}}{\partial \boldsymbol{a}_{\circ,k}} = \frac{\partial r(\boldsymbol{s}_{\circ,k}, \boldsymbol{a}_{\circ,k})}{\partial \boldsymbol{a}_{\circ,k}} + \boldsymbol{\lambda}_{\circ,k+1}^\top \frac{\partial F_\circ(\boldsymbol{s}_{\circ,k}, \boldsymbol{z}_k, \boldsymbol{a}_{\circ,k}; \boldsymbol{\theta}_p)}{\partial \boldsymbol{a}_{\circ,k}}. \tag{42}$$

Differentiating the Hamiltonian (32) with respect to the actions should now make the connection

$$\frac{\partial Q_{\circ,k}}{\partial \boldsymbol{a}_{\circ,k}} \equiv \frac{\partial \mathcal{H}_k}{\partial \boldsymbol{a}_{\circ,k}}, \tag{43}$$

that is, the gradient of the twin-induced Q-function with respect to the action coincides with the gradient of the Hamiltonian with respect to the action.

It is important to stress that coincidence of the action sensitivities in (43) does not imply that the Hamiltonian and the Q-function are identical objects. The former depends explicitly on the costate variable, whereas the latter embeds the full cost-to-go. What coincides is their first-order sensitivity with respect to the action when the costate is identified with the gradient of the value function.

5.3 Reciprocal Reinforcement

The symmetry highlighted in the previous section shows that the critic is to the model-free actor what the digital twin is to the model-based actor: both provide the sensitivity of cumulative reward with respect to the action, and thereby define the descent direction of the policy parameters. Within the general reinforcement twinning framework of Figure 4, this symmetry can be leveraged to promote reciprocal reinforcement between the model-based policy $\pi_\circ(\cdot \mid \boldsymbol{\theta}_{\pi,\circ})$ and the model-free policy $\pi_\bullet(\cdot \mid \boldsymbol{\theta}_{\pi,\bullet})$. The former is primarily trained from virtual interactions using model-based sensitivities, while the latter is primarily trained from real transitions through actor–critic learning.

Nevertheless, their training processes are intimately linked within the overall architecture. The subscripts $\circ$ and $\bullet$ therefore identify not merely two parametrizations, but two learning worlds: one governed by the predictive structure of the twin, the other by the empirical value landscape inferred from data. Both policies are advanced in parallel, and their interaction is mediated by the executed trajectories and the replay buffer.

Each branch brings complementary strengths and weaknesses.

The model-based branch ($\circ$) benefits from explicit knowledge of the system dynamics. Through adjoint propagation, it produces low-variance and structurally precise gradient information, often leading to rapid convergence and high sample efficiency. However, its descent direction is only as reliable as the twin itself: model bias translates directly into gradient bias. Moreover, adjoint-based updates are inherently on-policy, tied to the current trajectory induced by the policy, and lack the distribution-smoothing mechanisms (such as replay buffers and off-policy sampling) that facilitate the training of highly expressive neural parametrizations.

Conversely, the model-free branch ($\bullet$) bypasses the need for an explicit model. By approximating the action–value function directly from data, it can in principle represent arbitrarily complex performance landscapes and learn policies that transcend hand-crafted parametrizations. Its off-policy nature enables the reuse of past transitions and stabilizes deep-network training. Yet learning the critic is notoriously difficult—as experienced in Tutorial 4 of Lecture 5—gradient estimates exhibit high variance, and large numbers of rollouts are typically required before a reliable value landscape emerges.

The structural complementarity of the two branches naturally gives rise to multiple mechanisms of reciprocal reinforcement, some of which are outlined below. Although hybridization between model-based and model-free updates has appeared in various forms, including Dyna-style architectures (Sutton, 1991), residual policy learning (Johannink et al., 2019a), and model-based value expansion (Feinberg et al., 2018), meta-policy optimization (Clavera et al., 2018) or stochastic ensemble value expansion (Buckman et al., 2018), the mechanisms proposed below, building on the Hamiltonian–Q symmetry.

Hybrid gradient exchange. The descent directions computed in the two worlds could be blended. For example, the model-based policy may incorporate critic information extracted from real transitions,

$$\nabla_{\boldsymbol{\theta}_{\pi,\circ}} \mathcal{R}_{\text{hybrid}} = (1-\eta)\, \nabla_{\boldsymbol{\theta}_{\pi,\circ}} \mathcal{R}_{A,\circ} + \eta\, \mathbb{E}\Big[\nabla_{\boldsymbol{\theta}_{\pi,\circ}} \pi_\circ(\boldsymbol{o})\, \nabla_{\boldsymbol{a}} Q_\bullet(\boldsymbol{o},\boldsymbol{a}) \Big|_{\boldsymbol{a}=\pi_\circ(\boldsymbol{o})} \Big], \tag{44}$$

with $\eta \in [0,1]$. When $\eta = 0$ the update is purely adjoint-based; when $\eta = 1$ it is entirely critic-driven. Intermediate values allow the twin to benefit from real-data value information whenever model bias distorts its gradient field.

Twin-induced value shaping. The symmetry also suggests the reverse exchange. Since the twin defines its own state–action value function $Q_\circ$ through

$$Q_{\circ,k}(\boldsymbol{s}_{\circ,k}, \boldsymbol{a}_{\circ,k}) = r(\boldsymbol{s}_{\circ,k}, \boldsymbol{a}_{\circ,k}) + V_{\circ,k+1}(F_\circ(\boldsymbol{s}_{\circ,k}, \boldsymbol{z}_k, \boldsymbol{a}_{\circ,k}))\,,$$

simulated rollouts can be used to shape or initialize the critic learning in the real world.

Concretely, virtual transitions $(\boldsymbol{o}_{\circ,k}, \boldsymbol{a}_{\circ,k}, r_{\circ,k}, \boldsymbol{o}_{\circ,k+1})$ may be injected into the replay buffer with controlled weights, or used to pre-train the critic parameters $\boldsymbol{\theta}_{\bullet Q}$ before large-scale experimentation. This strategy aligns with classical Dyna-style and model-based value-expansion approaches, in which models are used to generate return estimates. Within the present framework, however, these rollouts can be interpreted as an explicit construction of the twin-induced Q-function, thereby making the Hamiltonian–Q symmetry operational. This symmetry could be pushed further to construct hybrid auxiliary

targets of the form

$$y_k^{\mathrm{hyb}} = (1-\alpha)\, y_k^{\bullet} + \alpha\, Q_{\circ,k}(\boldsymbol{o}_{\circ,k}, \boldsymbol{a}_{\circ,k}),$$

blending real-data Bellman targets with twin-based value predictions.

In this way, the twin acts as a structured prior on the value landscape: physics informs value approximation where data are scarce, while real transitions progressively correct model-induced bias. Rather than replacing data-driven learning, the twin reduces its statistical burden.

Distribution shaping through arbitration. A third mechanism of reciprocal reinforcement operates indirectly, through the referee. At each time step, only one action can be applied to the real system,

$$\boldsymbol{a}_k = \Pi_k(\pi_\circ(\boldsymbol{o}_k), \pi_\bullet(\boldsymbol{o}_k))\,,$$

where Π_k denotes the arbitration rule. This choice determines which policy generates the transition $(\boldsymbol{o}_{\bullet,k}, \boldsymbol{a}_k, r_k, \boldsymbol{o}_{\bullet,k+1})$ that populates the real replay buffer $\mathcal{D}_\bullet$.

Since the model-free critic $Q_\bullet$ is trained from samples drawn from $\mathcal{D}_\bullet$, the referee effectively controls the empirical state–action distribution $d^\pi(\boldsymbol{s}, \boldsymbol{a})$ under which value approximation is performed. If $\pi_\circ$ dominates execution, then $\mathcal{D}_\bullet \sim d^{\pi_\circ}$, and the critic is trained primarily on trajectories induced by the model-based branch. In this regime, the model-free actor is implicitly exposed to structured demonstrations, as the learned value landscape reflects the regions of state space explored by $\pi_\circ$.Conversely, if $\pi_\bullet$ dominates execution, $\mathcal{D}_\bullet \sim d^{\pi_\bullet}$, the critic learns from exploratory trajectories, and the twin is indirectly driven toward regions of the state space discovered through data-driven exploration. Since the identification dataset $\mathcal{D}_T^{(e)} = \{\boldsymbol{o}_{\bullet,k}, \boldsymbol{a}_k, \boldsymbol{z}_k\}_{k=1}^{n_t}$ depends on the executed policy, arbitration also shapes the distribution under which twin parameters $\boldsymbol{\theta}_p$ are updated.

The referee therefore acts as a distribution-shaping operator: it does not modify gradients explicitly, but determines which gradients will dominate subsequent learning by selecting the trajectories that define both $\mathcal{D}_\bullet$ and $\mathcal{D}_T$. In this sense, arbitration is itself a learning mechanism, governing the statistical coupling between the two branches.

Several arbitration principles are considered below.

(i) Model-based arbitration (twin-driven). Since both candidate policies can be evaluated inside the digital twin, the referee may compare their predicted cumulative rewards, $\mathcal{R}_{A,\circ}^{(\circ)}$ and $\mathcal{R}_{A,\circ}^{(\bullet)}$, obtained by rolling out $\pi_\circ$ and $\pi_\bullet$ within the twin. The executed action is selected as

$$\boldsymbol{a}_k = \begin{cases} \pi_\circ(\boldsymbol{o}_k), & \mathcal{R}_{A,\circ}^{(\circ)} \geq \mathcal{R}_{A,\circ}^{(\bullet)}, \\ \pi_\bullet(\boldsymbol{o}_k), & \text{otherwise.} \end{cases}$$

This strategy, adopted in Schena et al. (2024); Poletti et al. (2025), bases arbitration entirely on model predictions and is reliable only insofar as the twin provides sufficiently accurate performance estimates. This strategy is appropriate when the digital twin provides sufficiently accurate short-horizon performance predictions and reliable sensitivity information. In such regimes, twin-driven arbitration enables sample-efficient learning and protects the system from unnecessary exploration in sensitive operational conditions.

(ii) Value-based arbitration (critic-driven).

At the opposite extreme, arbitration may rely exclusively on information gathered from real interactions. The critic $Q_\bullet$ is trained from transitions stored in the replay buffer $\mathcal{D}_\bullet = \{(\boldsymbol{o}_{\bullet,k}, \boldsymbol{a}_k, r_k, \boldsymbol{o}_{\bullet,k+1})\}_{k=1}^{N_{\text{tot}}}$. The referee compares the averaged action–value predictions over a mini-batch $\{\boldsymbol{o}_\bullet^{(i)}\}_{i=1}^{N} \subset \mathcal{D}_\bullet$,

$$\overline{\Delta Q} = \frac{1}{N}\sum_{i=1}^{N}\Big[Q_\bullet\Big(\boldsymbol{o}_\bullet^{(i)}, \pi_\bullet(\boldsymbol{o}_\bullet^{(i)})\Big) - Q_\bullet\Big(\boldsymbol{o}_\bullet^{(i)}, \pi_\circ(\boldsymbol{o}_\bullet^{(i)})\Big)\Big]. \tag{45}$$

The policy associated with the larger averaged value is selected. Arbitration is therefore governed entirely by the empirical value landscape inferred from real data, without recourse to model predictions. This strategy is appropriate when the critic has converged to a reliable approximation of the empirical value landscape and model bias is suspected to distort twin-based predictions. In this regime, arbitration grounded in real data prevents systematic propagation of modeling errors.

(iii) Confidence-weighted blending.

Rather than switching abruptly between policies, the referee may combine them through a convex interpolation,

$$\boldsymbol{a}_k = (1-\zeta_k)\,\pi_\circ(\boldsymbol{o}_k) + \zeta_k\,\pi_\bullet(\boldsymbol{o}_k),$$

where $\zeta_k \in [0,1]$ quantifies relative confidence in the two learning branches and may be constructed from quantitative indicators of reliability, such as critic uncertainty, temporal-difference errors, ensemble variance in $Q_\bullet$, or the twin prediction residual $\ell_{\text{pred}}(\boldsymbol{o}_{\bullet,k}, \boldsymbol{o}_{\circ,k})$. A simple realization may balance critic uncertainty and twin prediction residual, for example as follows

$$\zeta_k = \frac{\sigma_Q^2(\boldsymbol{o}_{\bullet,k})}{\sigma_Q^2(\boldsymbol{o}_{\bullet,k}) + \alpha\,\ell_{\text{pred}}\Big(\boldsymbol{o}_{\bullet,k}, \boldsymbol{o}_{\circ,k}\Big) + \varepsilon}, \tag{46}$$

where σ_Q^2 denotes an estimate of critic variance (e.g. from an ensemble or bootstrap critic), ℓ_{pred} is the twin prediction residual defined in (9), with $\alpha > 0$ balances their relative scales, and $\varepsilon > 0$ ensures numerical robustness.

Large model residuals increase ζ_k, shifting authority toward the data-driven branch, whereas high critic uncertainty reduces ζ_k, restoring reliance on the twin. In the limits $\zeta_k = 0$ and $\zeta_k = 1$, the architecture reduces to purely model-based and purely model-free control, respectively. Intermediate values enable a continuous interpolation between the two learning worlds, shaping the state–action distribution without abrupt switching.

Such blended arbitration is particularly appropriate in transitional regimes where neither the twin nor the critic can be fully trusted, allowing authority to migrate smoothly as confidence evolves.

5.4 A note on the "control to modeling" mode

If the twin captures all the relevant dynamics and the closure parameters converge to $\boldsymbol{\theta}_p^\star$ such that

$$F_\circ(\boldsymbol{s}, \boldsymbol{z}, \boldsymbol{a}; \boldsymbol{\theta}_p^\star) = F_\bullet(\boldsymbol{s}, \boldsymbol{z}, \boldsymbol{a})$$

for all admissible states and actions, then for any policy π, $\mathcal{R}_{A,\circ}(\pi) = \mathcal{R}_{A,\bullet}(\pi)$, and the adjoint gradient satisfies $\nabla_{\boldsymbol{\theta}_{\pi,\circ}}\mathcal{R}_{A,\circ} = \nabla_{\boldsymbol{\theta}_{\pi,\circ}}\mathcal{R}_{A,\bullet}$.

In this regime the model-based gradient is exact, the referee becomes inactive, and the architecture collapses to classical adjoint-based optimal control. The model-free branch adds no new information and eventually aligns with the model-based solution. This proves the consistency property: the modeling to control operation of reinforcement twinning should reduce to classical model-based control when the model is perfect. The opposite extreme requires more attention: if the twin is consistently biased and the referee never elevates the model-based policy to interact with the real system, Reinforcement Twinning effectively reduces to standard actor–critic learning in the real environment.

In this condition, depending on operational constraints, the architecture may deliberately switch into the *control-to-modeling* regime illustrated in Figure 3, whereby the reward functional is modified and the objective shifts from performance optimization to model improvement.

The simplest (but not the only!) adaptation of the stage reward is

$$r_{\bullet,k} = -\,\ell_{\mathrm{pred}}\Big(\boldsymbol{o}_{\bullet,k}, \boldsymbol{o}_{\circ,k}(\boldsymbol{\theta}_p)\Big), \tag{47}$$

where ℓ_{pred} is the stage loss in (9). The corresponding cumulative reward becomes

$$\mathcal{R}^{(e)}_{A,\bullet}\Big(\boldsymbol{\theta}_{\pi,\bullet} \mid \boldsymbol{\theta}_p\Big) = -\mathcal{J}^{(e)}_T\Big(\boldsymbol{\theta}_p \mid \boldsymbol{\theta}_{\pi,\circ}, \boldsymbol{\theta}_{\pi,\bullet}\Big). \tag{48}$$

In this regime, maximizing return is equivalent to minimizing model prediction error along the executed trajectory.

The actor gradient therefore becomes directly linked to the sensitivity of the identification functional with respect to the applied actions. Since $\mathcal{R}^{(e)}_{A,\bullet} = -\mathcal{J}^{(e)}_T$, the deterministic policy gradient yields

$$\nabla_{\boldsymbol{\theta}_{\pi,\bullet}} \mathcal{R}^{(e)}_{A,\bullet} = -\sum_{k=0}^{n_t} \Big(\nabla_{\boldsymbol{\theta}_{\pi,\bullet}} \pi_\bullet(\boldsymbol{o}_{\bullet,k})\Big)^\top \nabla_{\boldsymbol{a}_k} \mathcal{J}^{(e)}_T. \tag{49}$$

Hence the critic-based update estimates the action sensitivity $\nabla_{\boldsymbol{a}_k} \mathcal{J}^{(e)}_T$, so that the policy parameters are adjusted in directions that increase the information content of the executed trajectory. The critic therefore approximates the action–value function associated with model improvement,

$$Q^\pi_\bullet(\boldsymbol{o}, \boldsymbol{a}) = \mathbb{E}\left[-\sum_{k=0}^{n_t-1} \ell_{\mathrm{pred}}\Big(\boldsymbol{o}_{\bullet,k}, \boldsymbol{o}_{\circ,k}(\boldsymbol{\theta}_p)\Big) \;\middle|\; \boldsymbol{o}_{\bullet,0} = \boldsymbol{o},\ \boldsymbol{a}_0 = \boldsymbol{a},\ \pi_\bullet\right], \tag{50}$$

so that $Q_\bullet$ measures the expected reduction of model discrepancy induced by a given action. This quantity represents the *epistemic gain* that can be generated through interaction. At the same time, the twin parameters are updated through the adjoint gradient

$$\nabla_{\boldsymbol{\theta}_p} \mathcal{J}^{(e)}_T = \sum_{k=0}^{n_t} \Big(\nabla_{\boldsymbol{o}_{\circ,k}} \ell_{\mathrm{pred}}\Big)^\top \nabla_{\boldsymbol{\theta}_p} \boldsymbol{o}_{\circ,k}. \tag{51}$$

Equations (49) and (51) show that both the actor and the digital twin descend the same functional $\mathcal{J}^{(e)}_T$, but along complementary directions in parameter space. The actor modifies the trajectory through the action sensitivity $\nabla_{\boldsymbol{a}_k} \mathcal{J}^{(e)}_T$, thereby shaping the distribution of visited states and influencing the information content of the collected data,

while the adjoint modifies the model through $\nabla_{\theta_p} \mathcal{J}_T^{(e)}$, thereby correcting the predictive structure.

The reinforcement loop therefore alters the trajectory distribution in order to influence the modeling gradient itself. In this control-to-modeling regime, interaction is no longer used to improve performance directly, but to improve the identifiability of the model. This makes explicit the bidirectional interaction between control and modeling that characterizes Reinforcement Twinning.

6 Tutorial: Stabilizing a Floating Wind Turbine

Wind energy deployment is increasingly moving offshore, where deep-water sites preclude bottom-fixed structures and require floating wind turbines (FWTs). The dynamics of FWTs are dominated by strong coupling between wave-induced platform motion and rotor aerodynamics (Jonkman, 2007; Stockhouse et al., 2022), which can induce destabilizing effects such as negative damping (Larsen and Hanson, 2007; Lackner, 2009; Karikomi et al., 2015). Advanced control strategies—such as MPC (Chaaban and C-P., 2014), active platform actuation (Stockhouse et al., 2022), structural damping (Namik et al., 2013), and H_∞ control (Li and Gao, 2015)—have been proposed The reader is referred to Lecture 19 by F. Lemmer for a deeper discussion of these aspects. All these methods rely on simplified models whose accuracy may deteriorate in strongly off-design or stochastic conditions.

The challenge considered here is to mitigate stochastic wind and wave disturbances with strictly limited actuation authority. In the simplified configuration adopted for this tutorial—without active ballast or mooring control—stabilization relies solely on blade pitch and generator torque. Standard land-based, power-maximizing strategies perform poorly in this setting, often amplifying platform motion and structural loads. To isolate the essential mechanisms, we employ a deliberately simplified dynamical model. Real FWTs involve coupled aero-hydro-servo-elastic dynamics, multiple structural modes, nonlinear hydrodynamics, wake interactions, and complex sensing and actuation. Rather than reproducing this full complexity, the tutorial uses a minimal model that preserves the core stabilization challenge while remaining transparent enough to analyze learning dynamics.

Objective. This tutorial serves as a guided exploration of the modeling-to-control formalism within the reinforcement twinning framework. The exercise unfolds in three stages. First, we address model identification and model-based control using data collected from the real system under a baseline controller. Second, we exploit the identified (imperfect) model as a sandbox to accelerate the training of an actor–critic algorithm. Finally, both the model-based and model-free agents are deployed on the real system within the full reinforcement twinning architecture, allowing us to investigate their reciprocal reinforcement and the transition between predictive and data-driven learning.

6.1 Problem set and simulated "real" environment

The configuration of interest is illustrated in Figure 5. It consists of a floating wind turbine mounted on a semi-submersible platform. The turbine has a rotor of radius R

rotating at angular velocity Ω and a tower of height H_t. The FWOT is subjected to an incoming wind flow with hub-height velocity v_∞, as well as to incoming waves with height η_{waves} with respect to the equilibrium state.

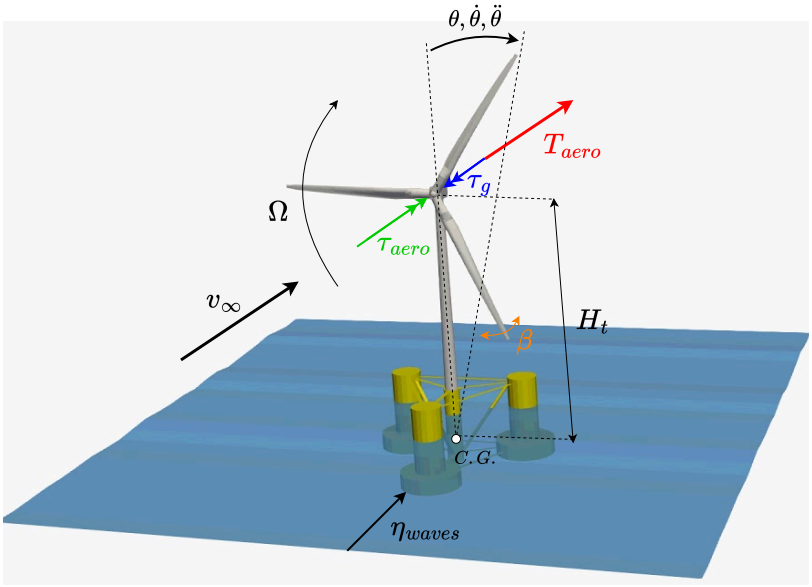


Figure 5: Configuration of interest and relevant variables.

A floating platform possesses six rigid-body degrees of freedom: surge, sway, and heave (translations), and roll, pitch, and yaw (rotations). In this tutorial, we restrict attention to a single degree of freedom, namely the pitch motion of the platform about its center of gravity (C.G.). This simplification captures the primary aero–hydro coupling mechanism responsible for negative damping effects, while keeping the dynamical model minimal.

The aerodynamic forces generated by the rotor produce an overall thrust force T_{aero}, acting approximately along the rotor axis, and an aerodynamic torque τ_{aero} acting on the low-speed shaft. The generator applies an opposing torque τ_g, while blade pitch control modifies the aerodynamic loading through the pitch angle β.

We model the distributed aerodynamic loading along the blades by an equivalent resultant thrust force T_{aero} acting at hub height. This resultant force generates a pitching moment about the platform C.G., thereby exciting the pitch motion. The corresponding pitch angle, angular velocity, and angular acceleration are denoted by θ, $\dot{\theta} = \omega$, and $\ddot{\theta} = \dot{\omega}$, respectively.

The pitch dynamics of the floating platform is modeled as a second–order nonlinear equation:

$$J_{\text{p}}\ddot{\theta} + D\dot{\theta} + D_v|\dot{\theta}|\dot{\theta} + K\theta = H_t T_{\text{aero}} + \tau_{\text{waves}}, \tag{52}$$

where J_{p} denotes the total rotational inertia about the pitch axis (including platform, turbine and added hydrodynamic inertia), D and D_v represent linear and quadratic hydrodynamic damping, and K is the hydrostatic restoring stiffness accounting for buoyancy and mooring effects. The aerodynamic thrust generates a pitching moment through the lever arm H_t, while τ_{waves} represents the external excitation due to waves.

Similar to tutorials 2 and 3 in lecture 5, we neglect drivetrain dynamics and blade aeroelasticity and model the rotor dynamics as a first order system

$$\dot{\Omega} = \frac{1}{J_{\text{rot}}} \left(\tau_{\text{aero}} - \tau_g \right), \tag{53}$$

with J_{rot} the rotor inertia. The aerodynamic torque is modeled using the power coefficient C_p as

$$\tau_{\text{aero}} = \frac{1}{2} \rho A \frac{C_p(\xi, \beta)}{\Omega} v^3, \tag{54}$$

where ρ is the air density, $A = \pi R^2$ is the rotor swept are, $\xi = \Omega R / v$ is the tip-speed ratio, and β is the blade pitch angle. Similarly, the aerodynamic thrust is obtained from the thrust coefficient C_T as:

$$T_{\text{aero}} = \frac{1}{2} \rho A C_T(\xi, \beta) v^2. \tag{55}$$

For a floating turbine, the effective wind speed seen by the rotor is modified by the platform motion:

$$v = v_\infty - H_t \dot{\theta}. \tag{56}$$

This kinematic coupling introduces an additional feedback between the aerodynamic and hydrodynamic subsystems and is the key mechanism responsible for potential negative damping. Both C_p and C_T are typically obtained using Blade Element Momentum Theory (BEMT). The ones used in this tutorial are shown in Figure 6. These were obtained with the `CCBlade` solver (Abbas et al., 2022) on a grid of values of β and ξ. The data on the grid were used to fit a surrogate model using ridge regression combining Gaussian radial basis functions (RBFs) in ξ and a polynomial basis in β as described in (Randino et al., 2025). This surrogate model is faster to evaluate than a local interpolator.

For the purposes of this exercise, we assume that the surrogate model for the C_p and C_T are accurate, that is the turbine dynamics is perfectly known.

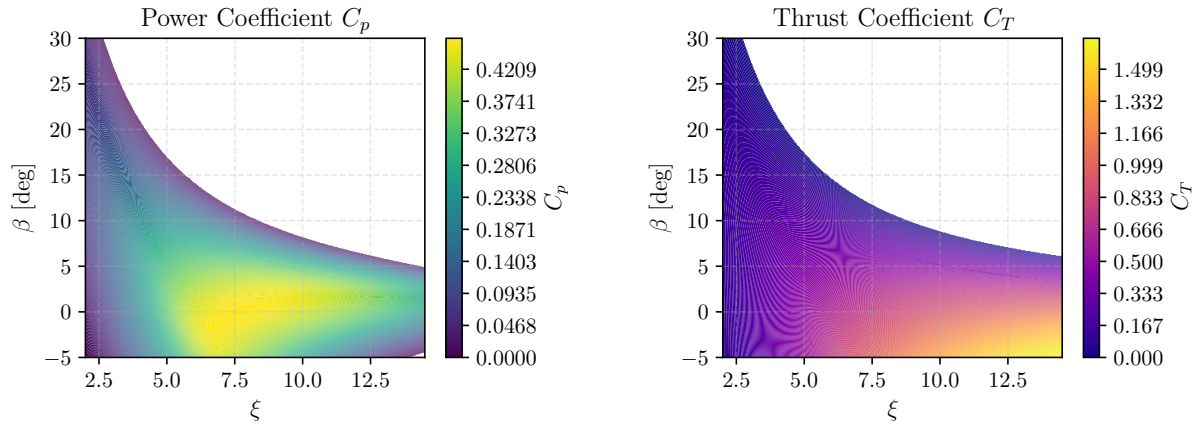


Figure 6: Contour plots of the power coefficient $C_p(\xi, \beta)$ and the thrust coefficient $C_T(\xi, \beta)$ for the turbine considered.

The actuator dynamics for blade pitch and generator torque are modeled as first-order systems, such that the commanded inputs $(\beta_c, \tau_{g,c})$ are not applied instantaneously.

Specifically,

$$\dot{\beta} = \frac{1}{\tau_\beta}\Big(\beta_c - \beta\Big), \qquad \dot{\tau}_g = \frac{1}{\tau_{\tau_g}}\Big(\tau_{g,c} - \tau_g\Big), \tag{57}$$

where τ_β and τ_{τ_g} are time constants representing the bandwidth of the pitch and torque actuators.

State–space formulation. Collecting the dynamics described above, we define the state vector

$$\boldsymbol{s} = \begin{bmatrix} \theta \\ \omega \\ \Omega \\ \beta \\ \tau_g \end{bmatrix}, \qquad \boldsymbol{a} = \begin{bmatrix} \beta_c \\ \tau_{g,c} \end{bmatrix}, \quad \boldsymbol{z} = \begin{bmatrix} v_\infty \\ \tau_{\text{waves}} \end{bmatrix} \tag{58}$$

where θ and $\omega = \dot{\theta}$ denote the platform pitch angle and angular velocity, Ω is the rotor speed, and (β, τ_g) are the actuator states associated with blade pitch and generator torque.

The closed-loop dynamics can then be written compactly as

$$\dot{\boldsymbol{s}} = F\Big(\boldsymbol{s}, \boldsymbol{z}, \boldsymbol{a}\Big), \tag{59}$$

with

$$F(\boldsymbol{s}, \boldsymbol{z}, \boldsymbol{a}) = \begin{bmatrix} \omega \\ \dfrac{1}{J_{\text{p}}}\Big(H_t T_{\text{aero}}(\Omega, \beta, v) + \tau_{\text{waves}} - D\,\omega - D_v|\omega|\omega - K\,\theta\Big) \\ \dfrac{1}{J_{\text{rot}}}\Big(\tau_{\text{aero}}(\Omega, \beta, v) - \tau_g\Big) \\ \dfrac{1}{\tau_\beta}\Big(\beta_c - \beta\Big) \\ \dfrac{1}{\tau_{\tau_g}}\Big(\tau_{g,c} - \tau_g\Big) \end{bmatrix}, \tag{60}$$

where the effective wind speed v seen by the rotor is defined in (56), the aerodynamic thrust and torque are obtained from the surrogate models $C_T(\xi, \beta)$ and $C_p(\xi, \beta)$ as described in the following.

This state–space model is used both to represent the real environment and to construct the digital twin. The two worlds differ only in the platform parameter vector $\boldsymbol{\theta}_p = [J_p, D, D_v, K]$. In the real environment these parameters take fixed (unknown) physical values $\boldsymbol{\theta}_p^\star$, whereas in the twin they are treated as trainable quantities to be identified from data.

Episodes are simulated over a horizon T with uniform time step Δt, resulting in $n_t = T/\Delta t$ transitions per episode. Time integration is carried out using a fourth-order Runge–Kutta (RK4) scheme.

Input Disturbances Both the inflow velocity and the wave excitation signals are modeled as Gaussian processes (GPs), such that

$$z_i(t) \sim \mathcal{GP}\Big(\mu_i(t), \kappa_i(t, t')\Big), \tag{61}$$

where $\mu_i(t)$ denotes the mean function and $\kappa_i(t,t')$ is the covariance kernel. This follows the framework introduced in Tutorials 2 and 3 of Lecture 5. The stochastic structure of both processes relies on a common two-scale covariance kernel, which captures variability occurring at two distinct temporal scales. This is defined as

$$\kappa_i^{\text{2-scales}}(t,t') = \sigma_{i,1}^2 \exp\left(-\frac{(t-t')^2}{2\,\ell_{i,1}^2}\right) + \sigma_{i,2}^2 \exp\left(-\frac{(t-t')^2}{2\,\ell_{i,2}^2}\right) + \sigma_{n,i}^2\,\delta(t-t'), \qquad i \in \{w,\eta\}, \tag{62}$$

where $\sigma_{i,1}$ and $\sigma_{i,2}$ denote the amplitude parameters associated with the two characteristic time scales $\ell_{i,1}$ and $\ell_{i,2}$, respectively, and $\sigma_{n,i}^2$ represents the variance of the white-noise component. The term $\delta(\cdot)$ denotes the Dirac delta function. The index $i = w$ refers to the wind generation case, whereas $i = \eta$ corresponds to the wave excitation case.

For the wave forcing, an additional harmonic kernel is introduced

$$\kappa_\eta(t,t') = \kappa_\eta^{\text{2-scales}}(t,t') + \sigma_h^2 \exp\left(-\frac{2}{\ell_H^2}\sin^2\left(\frac{\pi(t-t')}{T_\eta}\right)\right), \tag{63}$$

where σ_h controls the energy of the oscillatory component, T_w denotes the dominant wave period, and ℓ_H characterizes the temporal coherence of the wave train.

The selected Gaussian-process model enables faster generation of stochastic incoming disturbances than more realistic sea-state descriptions based on wave spectra (e.g. the JONSWAP spectrum in Mazzaretto et al. (2022)).

The kernel (63) is used to produce signals of wave elevation $\eta_w(t)$, while the wave torque that these apply to the platform can be estimated by convolving $\eta_w(t)$ with the hydrodynamic wave-excitation impulse response, typically computed for the floater using potential-flow BEM solvers (Robertson et al., 2014). For the purposes of this exercise, we omit these details and assume that the controller can be informed by the incoming waves and compute from these the associated moments τ_{waves} forcing the platform in (52).

Selected Conditions The selected wind turbine for this exercise is the NREL 5 MW reference model (Jonkman et al., 2009), characterized by a rotor radius of $R = 63$ m and a rotor inertia of $J_{\text{rot}} = 3.9 \times 10^7$ kg m^2. The system reaches its rated operating point at a wind speed of $v_\infty^{(r)} = 11.4$ m/s, corresponding to a rated low-speed-shaft (LSS) rotor speed of $\Omega^{(r)} = 1.3$ rad/s. The generator torque τ_g is expressed on the same shaft, allowing the gearbox ratio to be eliminated from the system equations.

The reduced one-degree-of-freedom pitch model of the platform, as described by Eq. (52), is parameterized with values representative of an OC4-type semi-submersible (Robertson et al., 2014). These coefficients represent the total inertia, hydrodynamic damping, nonlinear viscous effects, and hydrostatic restoring stiffness:

$$\boldsymbol{\theta}_p^\star = \begin{bmatrix} J_{\text{p}}^\star \\ D^\star \\ D_v^\star \\ K^\star \end{bmatrix} = \begin{bmatrix} 2.0 \times 10^9 \text{ kg m}^2 \\ 1.0 \times 10^7 \text{ N m s/rad} \\ 5.0 \times 10^7 \text{ N m s}^2\text{/rad}^2 \\ 2.0 \times 10^9 \text{ N m/rad} \end{bmatrix}. \tag{64}$$

These values are treated as unknown and needs to be identified in the model training phase. An instance of the digital twin is thus more generally encoded by a specific set of guess parameters $\boldsymbol{\theta}_p = [J_p, D, D_v, K]$.

Blade pitch and generator torque follow the actuator dynamics in (57), including magnitude saturation and rate limits representative of the NREL 5 MW turbine. The actuator time constants are $\tau_\beta = 1$ s, $\tau_g = 0.05$ s. These values are chosen such that the physical rate limits $|\dot{\beta}| \leq 8°/\text{s}$ and $|\dot{\tau}_g| \leq 15000$ kNm/s, are respected throughout all investigated operating conditions. The resulting actuation therefore remains within the admissible linear regime, without saturation effects. The actuation bounds are $\beta \in [0°, 45°]$ and $\tau_g \in [2150, 3940 \text{ kNm}]$. These limits are embedded in the actuation law, as discussed in the following.

Wind and wave signals The turbulent wind excitation ($i = w$) is generated using a kernel with parameters

$$[\sigma_{w,1}, \sigma_{w,2}, \sigma_{n,w}] = [1,\ 0.5,\ 0.1]\ \text{m}^2/\text{s}^2, \qquad [\ell_{w,1}, \ell_{w,2}] = [10,\ 0.5]\ \text{s}.$$

The mean wind speed is randomly selected in the range $\mu_w = 11$–13 m/s. These settings generate sufficiently large fluctuations for the instantaneous wind speed to repeatedly cross the rated threshold, driving the turbine operation across both sub-rated and above-rated regimes. For the wave excitation ($i = \eta$), the kernel parameters are

$$[\sigma_{\eta,1}, \sigma_{\eta,2}, \sigma_{n,\eta}, \sigma_h] = [0.7,\ 0.45,\ 0.03,\ 2]\ \text{m}^2,$$
$$[\ell_{\eta,1}, \ell_{\eta,2}, \ell_H] = [2,\ 1.5,\ 2]\ \text{s}, \qquad T_w = 10\ \text{s},$$

with zero mean, $\mu_\eta = 0$. An example signal for the inflow velocity, wave elevation and associated induced moment are shown in Figure 7.

Episode Initialization At the beginning of each episode, the initial state $\boldsymbol{s}_0$ is sampled from a multivariate Gaussian distribution centered at the equilibrium values:

$$\boldsymbol{s}_0 \sim \mathcal{N}\big(\bar{\boldsymbol{s}}(v_0, \tau_{w,0}), \boldsymbol{\Sigma}\big)\ , \tag{65}$$

where the reference state $\bar{\boldsymbol{s}} = [\theta^\star, \dot{\theta}^\star, \Omega^\star]^\top$ is determined by finding the steady-state fixed point of the system under the sampled environmental conditions v_0 and $\tau_{w,0}$, and the covariance matrix $\boldsymbol{\Sigma} = \text{diag}(\sigma_\theta^2, \sigma_{\dot{\theta}}^2, \sigma_\Omega^2)$ define the allowed variability with respect to the mean values. The initial state is sampled from a Gaussian distribution centered at the equilibrium and truncated to physically admissible bounds,

$$\theta_0 \in [-0.01,\ 0.01]\ \text{rad}, \qquad \dot{\theta}_0 \in [-0.01,\ 0.01]\ \text{rad/s}, \qquad \Omega_0 \geq 0.3\ \text{rad/s}. \tag{66}$$

We here consider $\sigma_\theta = 0.5°$, $\sigma_{\dot{\theta}} = 0.03$ rad/s, drawn from the expected operational dynamics of the OC4 (Robertson et al., 2014), and $\sigma_\Omega = 0.0$. For the above-rated conditions considered in this exercise, we enforce $\Omega^\star = \Omega^{(r)}$ and $\dot{\theta}^\star = 0$. The steady-state blade pitch angle $\beta^\star$ is obtained by solving the rotor torque balance at rated speed,

$$\tau_{\text{aero}}(v_0, \beta^\star, \Omega^{(r)}) = \tau_g^{(r)}, \tag{67}$$

where $\tau_g^{(r)}$ denotes the generator torque applied on the rotor (i.e. already accounting for the gearbox ratio). Given the equilibrium pitch $\beta^\star$, the steady-state platform tilt $\theta^\star$ is determined from static moment balance,

$$\theta^\star = \frac{T_{\text{aereo}}(v_0, \beta^\star, \Omega^{(r)})\, H_t + \tau_{w,0}}{K}, \tag{68}$$

where T is the rotor thrust, H_t the hub height, $\tau_{w,0}$ the external wave-induced disturbance torque, and K the hydrostatic restoring stiffness.

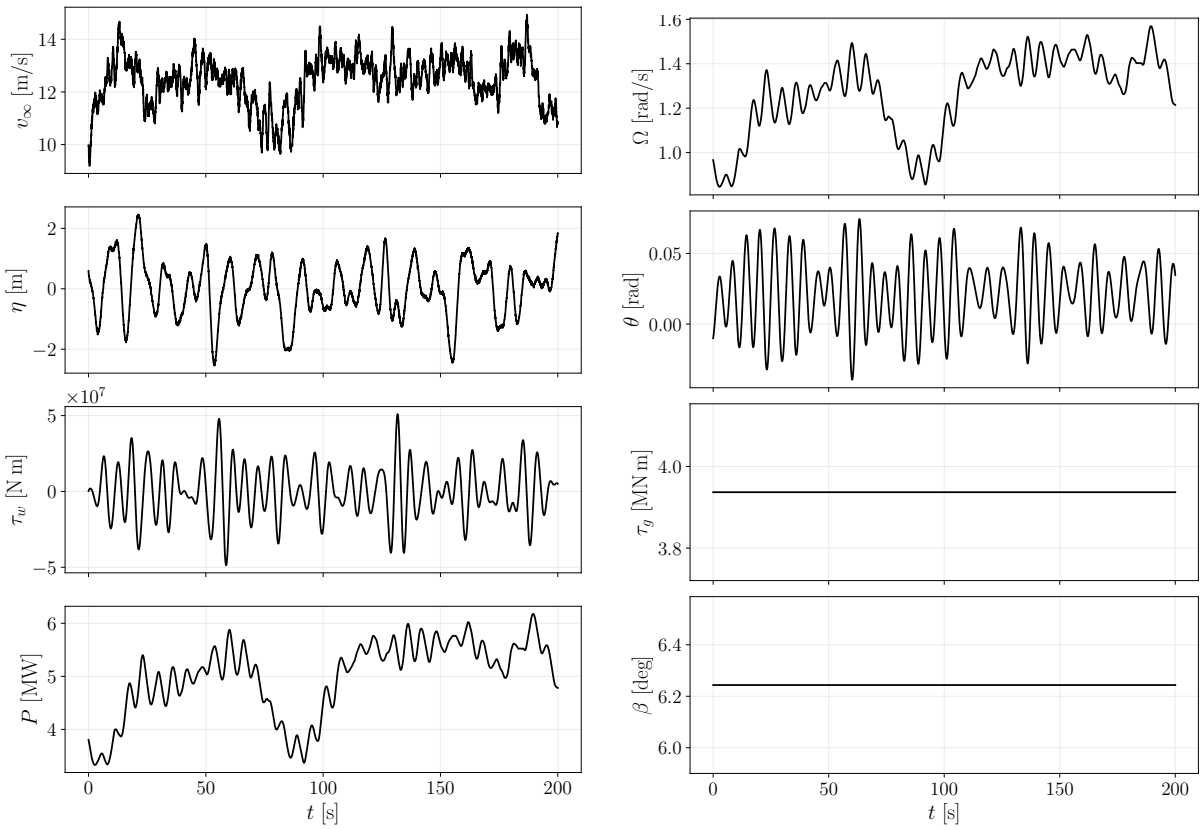


Figure 7: Representative closed-loop response under the baseline above-rated policy with constant generator torque and constant blade pitch. The left panel reports the main states and control inputs (rotor speed Ω, platform pitch θ, generator torque $\tau_g = \tau_g^{(r)}$, and blade pitch $\beta = \beta^\star$). The right panel shows the applied disturbances (turbulent wind speed v_∞, wave elevation η, and wave-induced pitch moment τ_w) together with the resulting electrical power P. Despite significant wind and wave variability, the open-loop baseline keeps the actuation fixed.

Baseline Controller Above rated wind speed, the wind turbine's goal shifts from increasing power to limiting it. The aerodynamic torque must be controlled so rotor speed and power remain near rated values while avoiding excessive structural loads. This is typically done by holding the generator torque near its rated value and using blade pitch to shed excess aerodynamic power (Jonkman, 2007).

Although the unsteady wind frequently crosses the rated threshold, we adopt as baseline a simplified above-rated policy that keeps both generator torque and blade pitch constant over each episode. This is clearly idealized: in practice, above-rated operation is usually managed by a pitch PI controller that regulates rotor speed around its rated value (Abbas et al., 2022). On floating turbines, however, this feedback loop can create effective negative aerodynamic damping and amplify platform pitch motion. Therefore, using constant generator torque and pitch offers a simple open-loop baseline that often substantially reduces platform motion under wave forcing. Specifically, the generator torque is fixed at the target value $\tau_g^{(r)}$, and the pitch command is set to the steady value

$\beta^\star$ from the above-rated equilibrium calculation (67). Figure 7 illustrates a representative response of the floating wind turbine to wind and wave disturbances with the baseline controller implementation.

Simulated sensor measurements and noise model. We assume that the controller has access to all quantities required to evaluate the policy in (74) and the reward in (70). In particular, the instantaneous electrical power $P_k = \tau_{g,k}\Omega_k$, the rotor speed Ω_k, and the platform pitch θ_k are available for feedback and reward assignement. To make the setting more realistic, these measurements are assumed to be corrupted by additive Gaussian noise and small episode-wise constant biases, mimicking sensor imperfections.

To make the setting more realistic, these measurements are corrupted by additive Gaussian white noise and small episode-wise constant biases. The standard deviations of the measurement noise are $\sigma_\theta = 2 \times 10^{-3}\,\mathrm{rad}$, $\sigma_\omega = 1 \times 10^{-2}, \mathrm{rad/s}$, $\sigma_\Omega = 2 \times 10^{-2}\,\mathrm{rad/s}$, while the episode-wise bias standard deviations are $\sigma_{\theta,\mathrm{bias}} = 5 \times 10^{-4}\,\mathrm{rad}$, $\sigma_{\omega,\mathrm{bias}} = 2 \times 10^{-3}\,\mathrm{rad/s}$, $\sigma_{\Omega,\mathrm{bias}} = 0$.

For the purpose of running simulations in the digital twin, the stochastic disturbances v_∞ and η are assumed to be known. This is a strong assumption: in practice, these quantities can only be inferred probabilistically, which would require ensemble averaging over many disturbance realizations and significantly increase computational cost. We deliberately avoid this additional layer of complexity to keep the tutorial accessible; the reader is referred to Schena et al. (2024) for a more general formulation.

Nevertheless, we refrain from using disturbance signals within the control law, thereby avoiding any feedforward strategy that would make the stabilization problem considerably easier.

Reward Shaping and Analysis The objective of this demonstration is to attenuate short–term power fluctuations induced by turbulent wind excitation and wave–driven platform motion, while maintaining operation near a reference condition. Here, this was prescribed as $P_{\mathrm{ref}} = 4\mathrm{MW}$ and a reference rotor speed $\Omega_{\mathrm{ref}} = 1.3\mathrm{rad/s}$. Given the large wind fluctuations and the wave forcing, these values conservatively seek to avoid to overload the turbine. The instantaneous electrical power is defined as

$$P_k = \tau_{g,k}\,\Omega_k. \tag{69}$$

The stage reward is defined as the negative of a weighted quadratic penalty,

$$\begin{aligned} r_k = -\Bigg[& w_P \left(\frac{P_k - P_{\mathrm{ref}}}{P_{\mathrm{ref}}}\right)^2 + w_{\Delta P}\, P_{\mathrm{hp},k}^2 + w_\theta\, \theta_k^2 \\ & + w_{\Delta\beta} \left(\frac{\beta_{c,k} - \beta_{c,k-1}}{\beta_{\max} - \beta_{\min}}\right)^2 + w_{\Delta\tau} \left(\frac{\tau_{g,c,k} - \tau_{g,c,k-1}}{\tau_{\max} - \tau_{\min}}\right)^2 \Bigg], \end{aligned} \tag{70}$$

where the subscript $(\cdot)_k$ denotes the value at time step k (i.e. $t_k = k\,\Delta t$). The performance metric is defined as the mean return per time step,

$$\mathcal{R} = \frac{1}{N} \sum_{k=0}^{N-1} r_k, \tag{71}$$

and the corresponding cost minimized during training is $J = -\mathcal{R}$, so that the objective remains independent of the episode length.

The first term, weighted by w_p, enforces regulation around the reference power P_{ref}. Its normalization by P_{ref} ensures scale invariance with respect to operating level. The second term, weighted by $w_{\Delta P}$, penalizes the high–frequency component of the normalized power signal,

$$P_{\text{lp},k} = (1-\alpha)\, P_{\text{lp},k-1} + \alpha\, P_k, \qquad \alpha = \text{clip}\left(\frac{\Delta t}{T_{\text{lp}}}, 0, 1\right), \tag{72}$$

$$P_{\text{hp},k} = \frac{P_k - P_{\text{lp},k}}{P_{\text{ref}}}, \tag{73}$$

where T_{lp} denotes the low–pass time constant. This filtering separates slow variations of the mean power level from rapid oscillations, so that the penalty acts primarily on wave–induced ripple rather than long–term operating shifts induced by wind variability.

The third term, weighted by w_θ penalizes platform pitch excursions, limiting the aerodynamic coupling between platform motion and rotor dynamics. The final two terms, weighted by $w_{\Delta\beta}$ and $w_{\Delta\tau}$ respectively, penalize variations of the commanded inputs, with distinct weights for pitch and generator torque, thereby controlling actuation variability in a way that reflects their different operational constraints and their different role in the stabilization.

The weights adopted in this demonstration are summarized in Table 3.

Table 3: Reward weights and filter parameter used in the demonstration.

w_P	$w_{\Delta P}$	w_Ω	w_θ	$w_{\Delta\beta}$	$w_{\Delta\tau}$	T_{lp} [s]
10	1000	0	0.01	10000	0.2	3

It is important to emphasize that stabilizing both the platform pitch θ and electrical power P are generally conflicting objectives. From (52), the wave-induced torque τ_{waves} directly drives platform pitch. Without feedforward cancellation of τ_{waves}, the only way to counter this disturbance is by changing aerodynamic thrust, which alters aerodynamic torque and thus electrical power. Conversely, one may attempt to keep the electrical power constant by adjusting generator torque while allowing larger platform motions. The reward structure therefore encodes an explicit trade-off between structural stabilization and power regulation.

The weight hierarchy reflects the main goal: attenuating short-term power ripple. The large $w_{\Delta P}$ heavily penalizes high-frequency power content, pushing the controller to suppress wave-induced oscillations. The smaller w_P regulates the mean power level but allows moderate deviations to reduce ripple. The small w_θ indicates that pitch stabilization is secondary to power quality in this experiment; pitch excursions are limited but tolerated to avoid strong power fluctuations.

The control increment penalties, $w_{\Delta\beta} \gg w_{\Delta\tau}$, follow from actuator roles. From (69), generator torque τ_g affects electrical power almost instantaneously, while blade pitch β affects it indirectly through rotor dynamics and is effectively low-pass filtered. It is therefore natural to let torque react quickly to short-term disturbances while enforcing smoother pitch changes. In this setup, generator torque mainly mitigates wave-induced

high-frequency ripple, whereas blade pitch handles slower variations due to changes in mean wind.

The low-pass time constant T_{lp} sets the frequency range classified as "ripple." With $T_{\mathrm{lp}} = 3\,\mathrm{s}$, oscillations typical of wave-excited platform motion are emphasized in the high-pass component and strongly penalized. Slower variations from wind fluctuations are treated as quasi-steady operating-point shifts and are mainly handled via the w_P term.

Overall, the weights are not chosen for full multi-objective optimality but to show how a structured reward can bias the policy toward a specific physical goal: reducing wave-induced power fluctuations while maintaining acceptable operating conditions.

6.2 Phase 1: Model identification and model-based control

This section makes use of the tools introduced in lecture 5 to identify a model from data (that is find the best possible set of parameters $\boldsymbol{\theta}_p = [J_{\mathrm{ptfm}},\, B,\, D_v,\, K]$)) and then synthesise a possible control policy from this model.

We assume that coarse engineering estimates of these parameters are available, although off by a large margin. We take $\boldsymbol{\theta}_p^{(o)} = [0.3J^{\star}_{\mathrm{ptfm}},\, 0.2B^{*},\, 0,\, 0.5K^{*}]$)). Since all parameters are strictly positive, we avoid constrained optimization or ad-ho projection by using a logarithmic transform

$$\boldsymbol{\ell}_{\theta_p} = \log(\boldsymbol{\theta}_p) = \Big[\log J_{\mathrm{ptfm}},\, \log B,\, \log D_v,\, \log K\Big],$$

so that the identification problem is reformulated as

$$\min_{\boldsymbol{\ell}_{\theta_p} \in \mathbb{R}^4} \; \mathcal{J}\Big(\boldsymbol{\theta}_p(\boldsymbol{\ell}_{\theta_p})\Big)\,,$$

and the physical parameters are reconstructed through the exponential map $\boldsymbol{\theta}_p = \exp\Big(\boldsymbol{\ell}_{\theta_p}\Big)$. This transformation guarantees $\boldsymbol{\theta}_p > 0$ without requiring explicit constraints. From a practical perspective, the optimizer updates $\boldsymbol{\ell}_{\theta_p}$ in $\mathbb{R}^4$, and the corresponding physical parameters are obtained at each iteration by exponentiation.

The dataset consists of 100 episodes generated from the real system operating under the baseline above-rated controller, where the generator torque is fixed at rated value and the blade pitch is set according to the steady above-rated equilibrium map. Cost function gradients are computed via reverse-mode automatic differentiation in JAX. The update is performed using a stochastic AMSGrad scheme with batches of 32 episodes and a learning rate of 0.02. Figure 8 reports the evolution of the stochastic identification loss over the AMSGrad iterations. The overall trend is clearly decreasing, confirming steady progress of the optimizer.

The final identified coefficients in Table 4 are compared with the ground-truth parameters used to generate the dataset. Platform inertia and restoring stiffness are captured with less than 10% error, but linear damping is underestimated and the nonlinear damping coefficient collapses to the small floor value imposed by the log-parameterization. Its effects are largely absorbed by the linear term. This suggests that the trajectories lack sufficient platform-rate amplitude and variability to reveal quadratic dissipation. Consequently, the identified model may be inaccurate in conditions where nonlinear damping dominates.

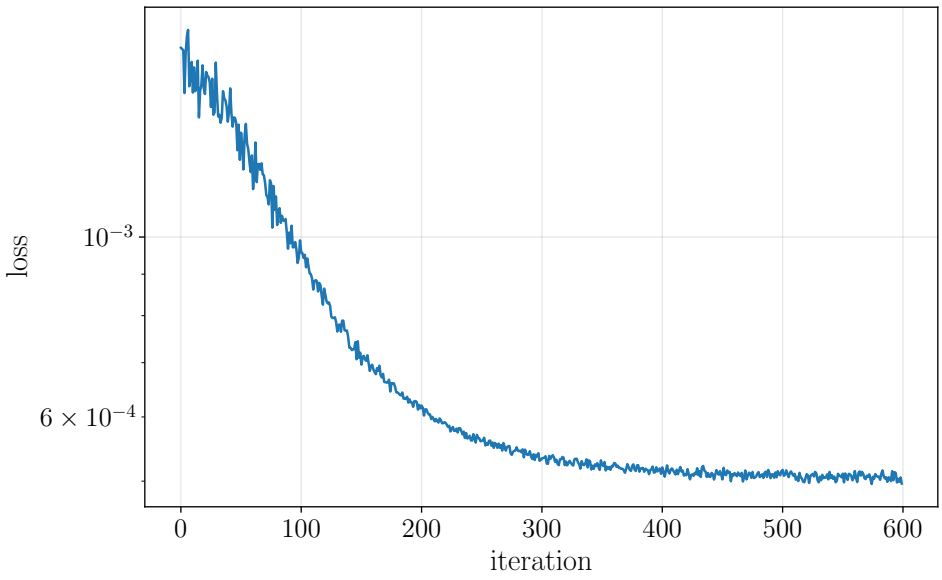


Figure 8: Convergence of the stochastic identification problem using AMSGrad. The loss exhibits mild fluctuations and occasional spikes due to mini-batch sampling, while the global decreasing trend indicates successful convergence.

Table 4: Identified floater parameters compared with the ground-truth values used in the data-generation model.

Parameter	Identified	Ground truth
J_{ptfm} [kg m^2]	$1.9105464e \times 10^9$	2.0×10^9
B [N m s/rad]	3.4796785×10^6	1.0×10^7
D_v [N m s^2/rad^2]	$9.99999931 \times 10^{-4}$	5.0×10^7
K [N m/rad]	1.9132268×10^9	2.0×10^9

Figure 9 shows an out-of-sample validation episode, generated with a fresh (previously unseen) wind–wave realization and corrupted by measurement noise and episode-wise sensor bias. In this setting, the identified model is simulated by replaying the *measured* inputs and initial condition, and its predictions are compared to the measured platform response. The agreement remains qualitatively correct, but noticeable discrepancies appear both in amplitude and phase of $\theta(t)$ and $\omega(t)$, indicating that the calibrated floater dynamics are still imperfect outside the training set.

Control Policy Once the model is available, the same adjoint-based stochastic gradient descent can be used to train a control policy. For the model-based actor, we consider the following static nonlinear state–feedback policy

$$\begin{bmatrix} \beta_{c,k} \\ \tau_{g,c,k} \end{bmatrix} = \boldsymbol{a}_{\text{mid}} + \boldsymbol{a}_{\text{amp}} \tanh\Big(\mathbf{K}\,\boldsymbol{\phi}_k\Big), \tag{74}$$

where $\mathbf{K} \in \mathbb{R}^{2\times 5}$ contains the policy parameters and $\tanh(\cdot)$ acts componentwise.

For consistency with the notation adopted throughout this chapter, we define the

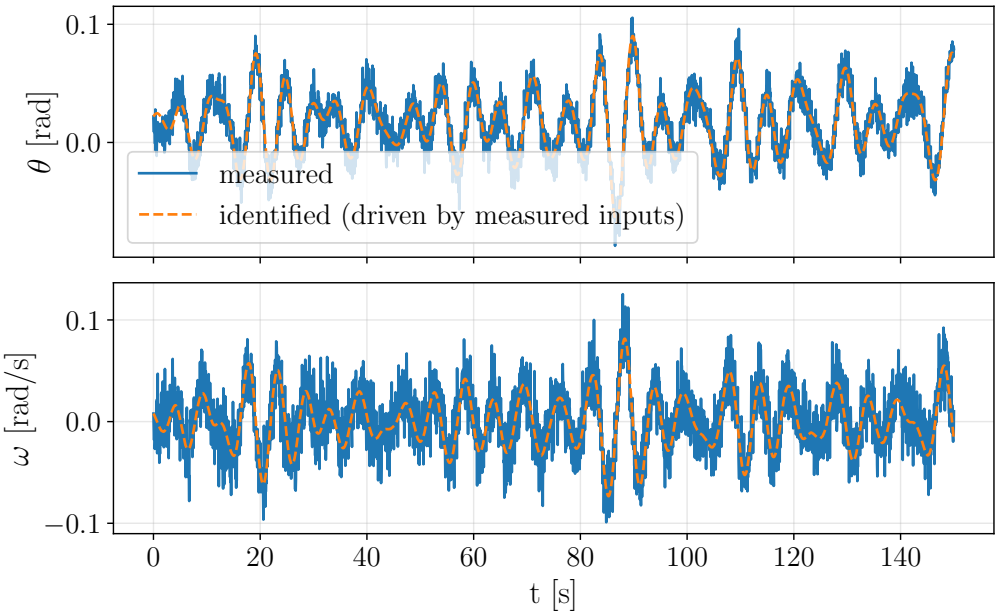


Figure 9: Out-of-sample validation episode with noisy measurements: comparison of measured platform pitch angle $\theta(t)$ and pitch rate $\omega(t)$ with predictions of the identified model driven by the measured inputs.

vector of model-based policy parameters as $\boldsymbol{\theta}_{\circ,\pi} := \mathrm{vec}(\mathbf{K})$, that is, $\boldsymbol{\theta}_{\circ,\pi}$ collects all entries of $\mathbf{K}$ into a single parameter vector.

The feature vector is defined as

$$\boldsymbol{\phi}_k = \left[1,\ \frac{\Omega_k - \Omega_{\mathrm{ref}}}{\Omega_{\mathrm{ref}}},\ \frac{P_k - P_{\mathrm{ref}}}{P_{\mathrm{ref}}},\ \frac{\theta_k - \theta_{\mathrm{ref}}}{\theta_{\mathrm{ref}}},\ \frac{\omega_k}{\omega_{\mathrm{ref}}}\right]^\top. \tag{75}$$

where $\Omega_{\mathrm{ref}} = 1.33$ rad/s, $P_{\mathrm{ref}} = 4$MW are taken from the target conditions, θ_{ref} is computed from the equilibrium condition and $\omega_{\mathrm{ref}} = 0.1$rad/s acts as a scaling value.

The constant feature allows the controller to shift the operating point. The normalized power deviation promotes regulation around P_{ref}, while the rotor speed term moderates kinetic–energy excursions. The platform angular velocity explicitly captures the dynamical pathway through which wave excitation modifies the relative inflow velocity and can induce negative aerodynamic damping. Including ω_k enables the policy to counteract this mechanism directly rather than responding only to its manifestation in the power signal.

The vectors

$$\boldsymbol{a}_{\mathrm{mid}} = \begin{bmatrix}(\beta_{\min} + \beta_{\max})/2 \\ (\tau_{\min} + \tau_{\max})/2\end{bmatrix}, \qquad \boldsymbol{a}_{\mathrm{amp}} = \begin{bmatrix}(\beta_{\max} - \beta_{\min})/2 \\ (\tau_{\max} - \tau_{\min})/2\end{bmatrix}, \tag{76}$$

map the bounded tanh output into the admissible actuator ranges. The commanded signals are subsequently passed through first–order actuator dynamics (57) with saturation, ensuring smooth evolution and physical feasibility of the applied inputs.

Model-based policy derivation Once the platform parameters have been identified, we proceed with the synthesis of the control policy (74) by maximizing the cumulative

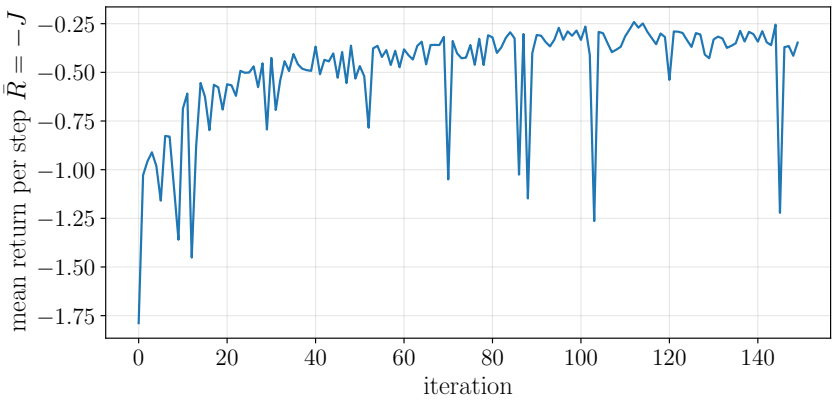


Figure 10: Evolution of the mean return per step $\bar{\mathcal{R}} = -J$ during model-based policy optimization. The policy is initialized to reproduce the baseline controller; improvements therefore correspond directly to performance gains over the baseline.

reward defined in (70)–(71). The optimization is performed directly on the policy gain matrix $\boldsymbol{K}$ using adjoint-based stochastic gradient descent, in exactly the same computational framework adopted for the identification stage.

The policy is initialized by regressing (74) on the baseline controller. In practice, the bias feature in the policy is chosen so that the initial control law exactly reproduces the constant baseline torque and pitch corresponding to the nominal equilibrium. As a consequence, any decrease in the objective function immediately corresponds to an improvement over the baseline behaviour, ensuring a fair comparison.

The optimization is carried out for 150 iterations, with a learning rate of 0.05 and a mini-batch of 10 disturbance realizations per gradient evaluation. Each batch consists of independently generated wind and wave episodes, and the policy is updated after each stochastic gradient step.

Figure 10 reports the evolution of the cumulative return per step $\mathcal{R}_{\circ,\pi}$ during training. The convergence curve exhibits the typical behaviour of stochastic gradient optimization in dynamical systems: an initial rapid improvement phase is followed by a progressively slower refinement. Small oscillations are visible throughout the iterations, reflecting the stochasticity induced by mini-batch sampling of disturbance realizations and the nonlinear nature of the closed-loop dynamics. Nevertheless, the overall trend is clearly monotonic in the envelope sense, and the adaptive learning-rate mechanism prevents divergence or large-amplitude instabilities. The final policy yields a substantial reduction of the objective compared to the baseline initialization.

Figure 11 compares the performances of the model-based policy against the baseline on the same wind and wave forcing realization. The optimized strategy maintains the electrical power significantly closer to the target value of 4 MW and largely attenuates the propagation of the wave forcing into the power output. Only low frequency excursions, primarily due to large scale wind variations are preserved.

The actuator signals reveal the expected mechanism promoted by the reward functional. The generator torque exhibits rapid variations that counteract the wave-induced disturbances, providing fast corrective action at short time scales, while the blade pitch angle evolves more slowly and with larger-amplitude excursions to adapt to the slowly

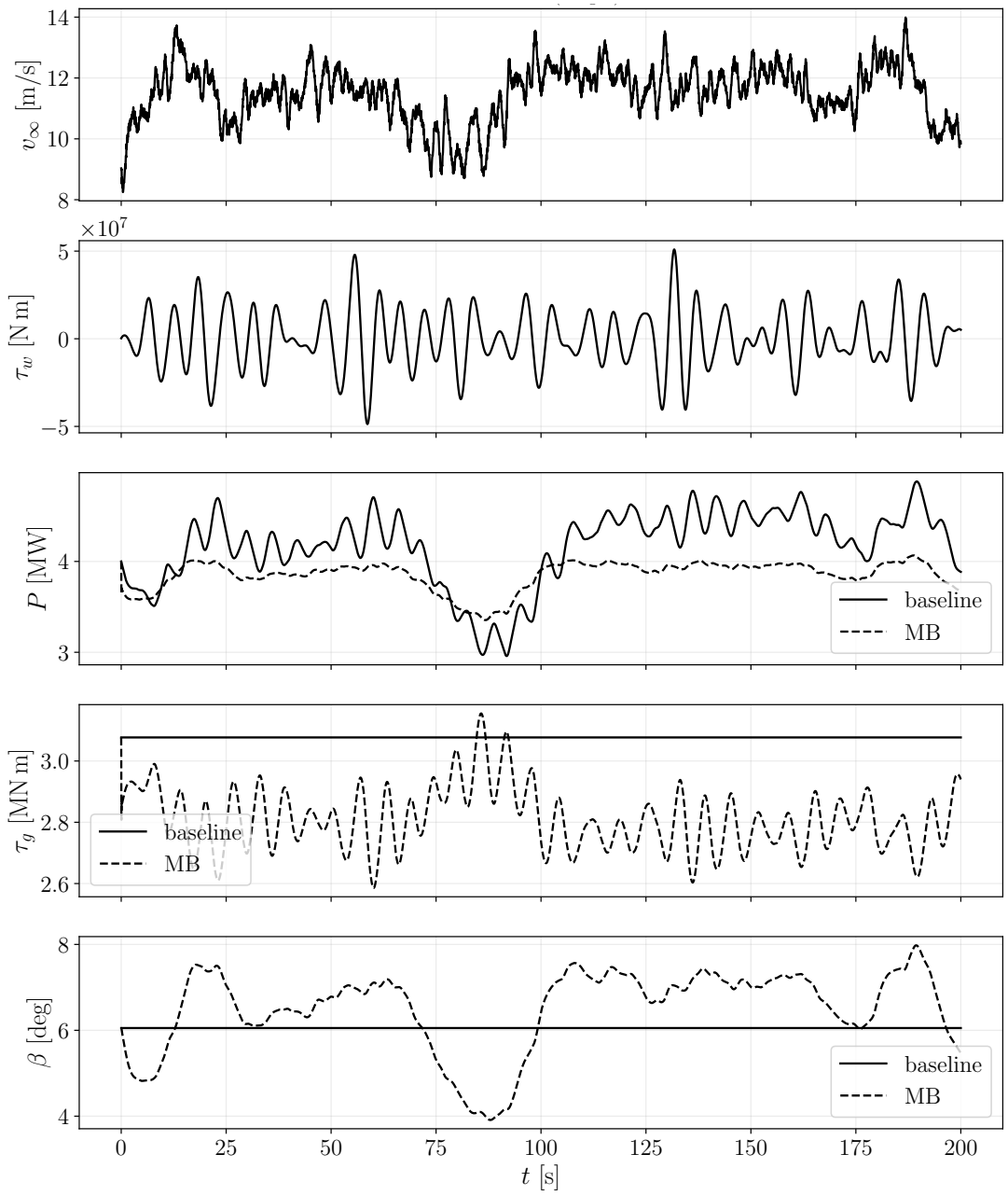


Figure 11: Closed-loop comparison under identical wind and wave disturbances. From top to bottom: wind speed v_∞, wave-induced torque τ_w, electrical power P, generator torque τ_g, and blade pitch angle β. Solid lines correspond to the baseline controller, dashed lines to the model-based (MB) policy.

varying wind conditions. This separation of time scales emerges naturally from the optimization and is not imposed explicitly, confirming that the reward structure successfully encodes the intended control strategy. Nevertheless, these performances are simulated on an imperfect model. We shall see soon what happens when deployes on the "real system". But first, let us train the other protagonist of this story and let the DDPG catch up with the derived policy.

The actuator signals reveal the expected mechanism promoted by the reward functional. The generator torque exhibits rapid variations that counteract the wave-induced disturbances, providing fast corrective action at short time scales, while the blade pitch angle evolves more slowly and with larger-amplitude excursions to adapt to the slowly varying wind conditions. This separation of time scales emerges naturally from the optimization and is not imposed explicitly, confirming that the reward structure successfully encodes the intended control strategy.

Table 5 quantifies the improvements observed in Figure 11. The dominant reduction stems from the high-pass filtered power term $w_{dP}\langle P_{\mathrm{hp}}^2\rangle$, confirming that the MB policy effectively suppresses wave-induced power oscillations. The tracking contribution $w_P\langle e_P^2\rangle$ is also significantly reduced, indicating tighter regulation around the 4 MW target. The additional actuator-variation penalties remain moderate, reflecting the intended separation of time scales between fast torque corrections and slower pitch adaptations. Overall, the model-based controller achieves an 83% improvement in cumulative reward under identical disturbances.

Table 5: Mean per-step contribution of the different terms of the reward function (70) under identical disturbances in the episode illustrated in Figure 11.

Term (mean per step)	Baseline	Model-based (MB)
$w_P\langle e_P^2\rangle$	1.213×10^{-1}	2.955×10^{-2}
$w_\Omega\langle e_\Omega^2\rangle$	0	0
$w_\theta\langle e_\theta^2\rangle$	6.012×10^{-3}	6.292×10^{-3}
$w_{dP}\langle P_{\mathrm{hp}}^2\rangle$	1.652	2.623×10^{-1}
$w_{d\beta}\langle(\Delta\beta)^2\rangle$	0	4.559×10^{-3}
$w_{d\tau}\langle(\Delta\tau_g)^2\rangle$	0	8.146×10^{-6}
Total reward $\mathcal{R} = -J$	-1.7788	$\mathbf{-0.3026}$ (**+83**%)

Although the gain is remarkable, these performances are obtained in simulation using an imperfect identified model. The real test comes when the policy is confronted with the "true" system. Before crossing that bridge, however, we allow the other protagonist of this story to enter the stage. We now pre-train a model-free controller on the available model and let a reinforcement learning agent attempt to match — or perhaps even outplay — the carefully engineered model-based policy derived above.

6.3 Phase 2: Model-Based Policy Optimization via RL

The control synthesis developed in Phase 1 (Section 6.2) leverages the identified dynamics to derive an optimal model-based policy, $\pi_\circ$. However, the optimality of this controller is inherently subordinate to its functional form, i.e. to the assumed parametrization used in its derivation. The present section therefore investigates whether a policy-agnostic representation can yield improved performance while still exploiting the model-based knowledge embedded in $\pi_\circ$. To this end, the analytical controller is used as a model-based prior to initialize and/or regularize a more flexible actor policy trained with a Twin Delayed Deep Deterministic Policy Gradient (TD3) framework (Fujimoto et al., 2018) over the surrogate dynamics identified in Phase 1. The identified model is recast into a Markov Decision

Process (MDP), in which each interaction step is represented by the transition tuple

$$\langle \boldsymbol{o}_k, \boldsymbol{a}_k, \boldsymbol{o}_{k+1}, r_k \rangle, \tag{77}$$

where $\boldsymbol{o}_k, \boldsymbol{o}_{k+1} \in \mathcal{O} \subseteq \mathbb{R}^{n_O}$ are the observations collected before and after actuation, $\boldsymbol{a}_k \in \mathcal{A} \subseteq \mathbb{R}^{n_A}$ is the continuous action, and $r_k \in \mathbb{R}$ is the reward (equivalently, the negative stage cost) accrued at step k. In this work, the observation vector is defined as

$$\boldsymbol{o}_k = \left[\frac{\Omega_k - \Omega_{\text{ref}}}{\Omega_{\text{ref}}}, \quad \frac{P_k - P_{\text{ref}}}{P_{\text{ref}}}, \quad \frac{\theta_k - \theta_{\text{ref}}}{\theta_{\text{ref}}}, \quad \omega_k - \omega_{\text{ref}}, \quad \tilde{\beta}_k, \quad \tilde{\tau}_k \right], \tag{78}$$

which extends the model-based counterpart in Eq. (75) by including the current actuator states β_k and τ_k. This extension is necessary because actuator first-order dynamics are explicitly modeled in the rollout loop, such that the applied inputs may lag behind the commanded ones for multiple integration steps; including the applied actuator states in the observation therefore helps preserve the Markov property of the transition dynamics. The actuator channels are encoded as normalized variables, $\tilde{\beta}_k$ and $\tilde{\tau}_k$, obtained by affine scaling of the physical variables from their admissible ranges $(\beta_{\min}, \beta_{\max})$ and $(\tau_{\min}, \tau_{\max})$ to the bounded interval $[-1, 1]$. The actor therefore maps $\boldsymbol{o}_k \in \mathbb{R}^6$ to continuous pitch and torque commands,

$$\boldsymbol{a}_{\bullet,k} = \pi_\bullet(\boldsymbol{o}_k \mid \boldsymbol{\theta}_{\pi,\bullet}), \qquad \boldsymbol{a}_{\bullet,k} = [\beta_{c,k}, \tau_{c,k}]^\top, \tag{79}$$

which are then applied through first-order actuator dynamics with $\tau_\beta = 1$ s and $\tau_{\tau_g} = 0.05$ s, subject to the bounds $\beta \in [0, 45]$ deg and $\tau_g \in [0.8, 1.2]\tau_{g,\text{rated}}$. The step reward is kept identical in structure to the model-based counterpart described in Eq. (70), so that any performance difference can be attributed to policy flexibility rather than to a modified objective. In particular, the reward is a negative weighted sum of normalized penalties on power tracking, rotor-speed tracking, platform-pitch tracking, high-pass power ripple, and action smoothness, with the action-variation terms computed on normalized command increments so that pitch and torque variations contribute on comparable dimensionless scales before weighting. The reference values are set to $P_{\text{ref}} = 4$ MW, $\Omega_{\text{ref}} = 1.3$ rad/s, and $\omega_{\text{ref}} = 0.1$ rad/s, while $\theta_{\text{ref}} = \theta^\star$ is obtained from a local equilibrium condition queried at the current operating condition. In the TD3 optimization runs reported here, the weights are tuned to

$$(w_P, w_\Omega, w_{\Delta P}, w_\theta, w_{\dot{\beta}}, w_{\dot{\tau}}) = (40.0,\ 3.0,\ 80.0,\ 0.01,\ 20.0,\ 0.1),$$

with $T_{P,\text{FILT}} = 3$ s for the low-pass/high-pass decomposition of power. These action-smoothness penalties remain critical to enforce operationally realistic control trajectories and to prevent the agent from exploiting nonphysical strategies that may artificially improve the surrogate reward (for instance, aggressive pitch transients used to compensate past-wave effects in a manner that would destabilize continuous operation). Each virtual episode is initialized from the same random initial-condition distribution defined in Eq. (65), and all rollouts use the same simulation horizon and integration step, here set to $\Delta t = 0.05$ s and 4000 steps per episode ($K_{\text{dec}} = 1$, i.e. one decision per integration step), under stochastic wind and wave disturbances sampled online from the Gaussian-process

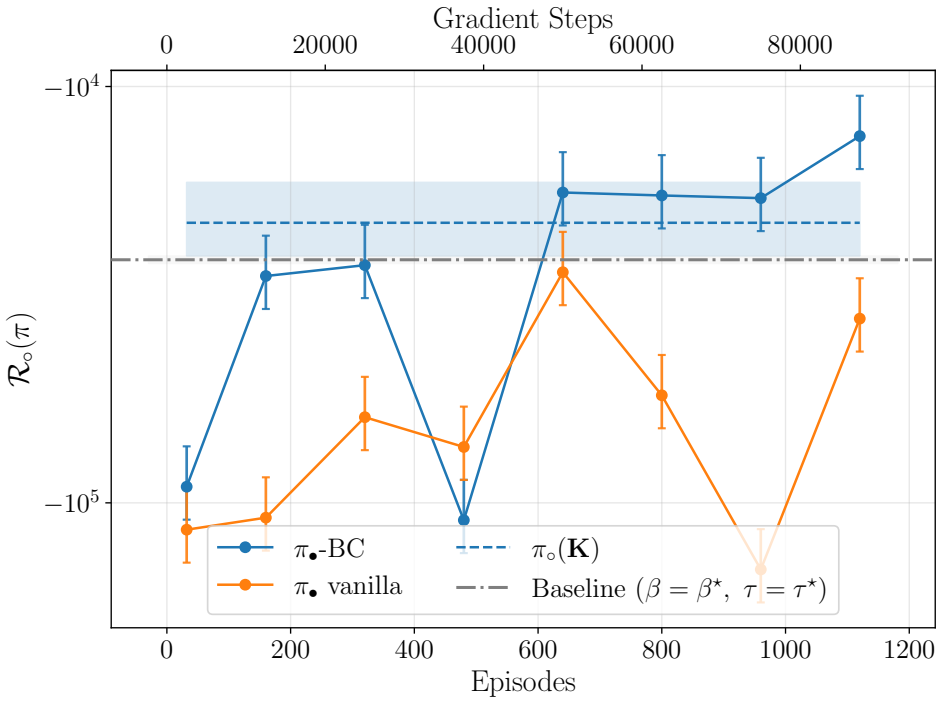


Figure 12: Learning curves of RL-based policies, with behavioral cloning ($\pi_\bullet$-BC) and without any imitation reward ($\pi_\bullet$-vanilla), compared against baseline controllers. Each agent is evaluated for $M = 30$ episodes in deterministic mode (no noise over actions, sensor noise and random disturbances are kept as in training conditions).

generators and with additive measurement noise injected during training and evaluation for robustness.

The actor and both critics are implemented as fully connected MLPs with two hidden layers of width 128 (actor input dimension 6, critic input dimension 8), the actor output layer is initialized with a small final scale (10^{-2}) and biased toward a nominal above-rated operating action, and optimization uses Adam with learning rates 5×10^{-5} (actor) and 5×10^{-4} (critics), discount factor $\gamma = 0.995$, soft target-update coefficient $\tau = 0.005$, batch size 2048, global gradient clipping at 1.0, and delayed actor updates (one actor step every two critic updates). Experience is stored in a prioritized replay buffer of capacity 5×10^6 transitions. In addition, expert-data injection from the stochastic model-based controller family is used during data collection (initial probability 0.3, decayed over training) to improve early replay-buffer coverage around stabilizing trajectories. The implementation supports behavioral cloning regularization through (26). Training is organized in 50 outer iterations, each with parallel collection of 32 episodes followed by 2500 gradient updates, and deterministic evaluation (target actor, no exploration noise) is performed every 5 iterations over 5 episodes. Finally, the training in this phase is carried out sampling the disturbances in the same range of what done for the model-based part (Phase 1, Sect. 6.2), i.e. $\bar{v}_\infty \in [11.0, 13.0]$ m/s, and wave period $T_w = 10$ s.

Following the experimental setup, the agents are evaluated to assess their learning dynamics and resulting closed-loop performance. The outcomes of this initial training stage are captured in Figure 12, which compares the learning curves of the proposed

actor–critic formulations against the baseline methods. As illustrated, a standard reinforcement learning approach (RL-vanilla) fails to acquire a stabilizing policy within the given training horizon, remaining trapped in a sub-optimal performance regime[1]. Integrating a behavioral cloning regularization term (RL-BC, Eq. (25)) effectively guides the agent toward good regions of the state space; eventually leading to a better policy than both baseline and model-based ones. In this study, we found that the λ_{BC} term in the actor loss (Eq. (25)) must be scheduled and reduced as the training progresses, to avoid stagnation, however, relative learning curves are not shown here for conciseness. The

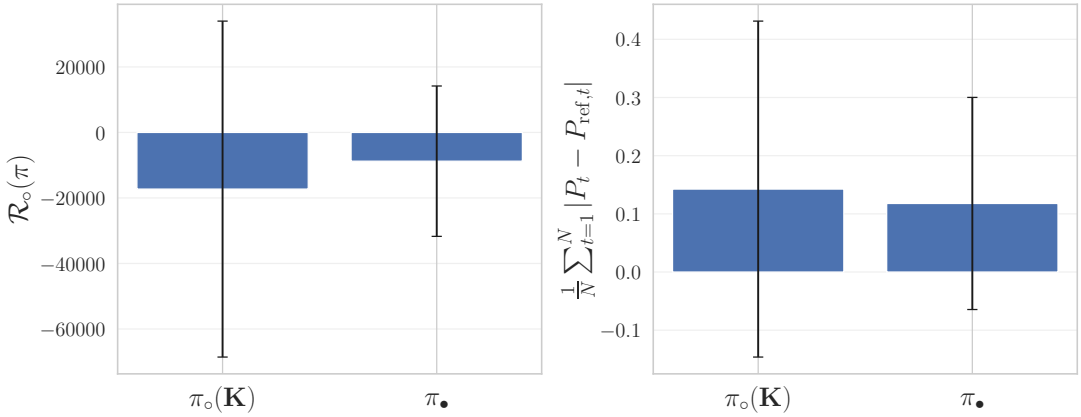
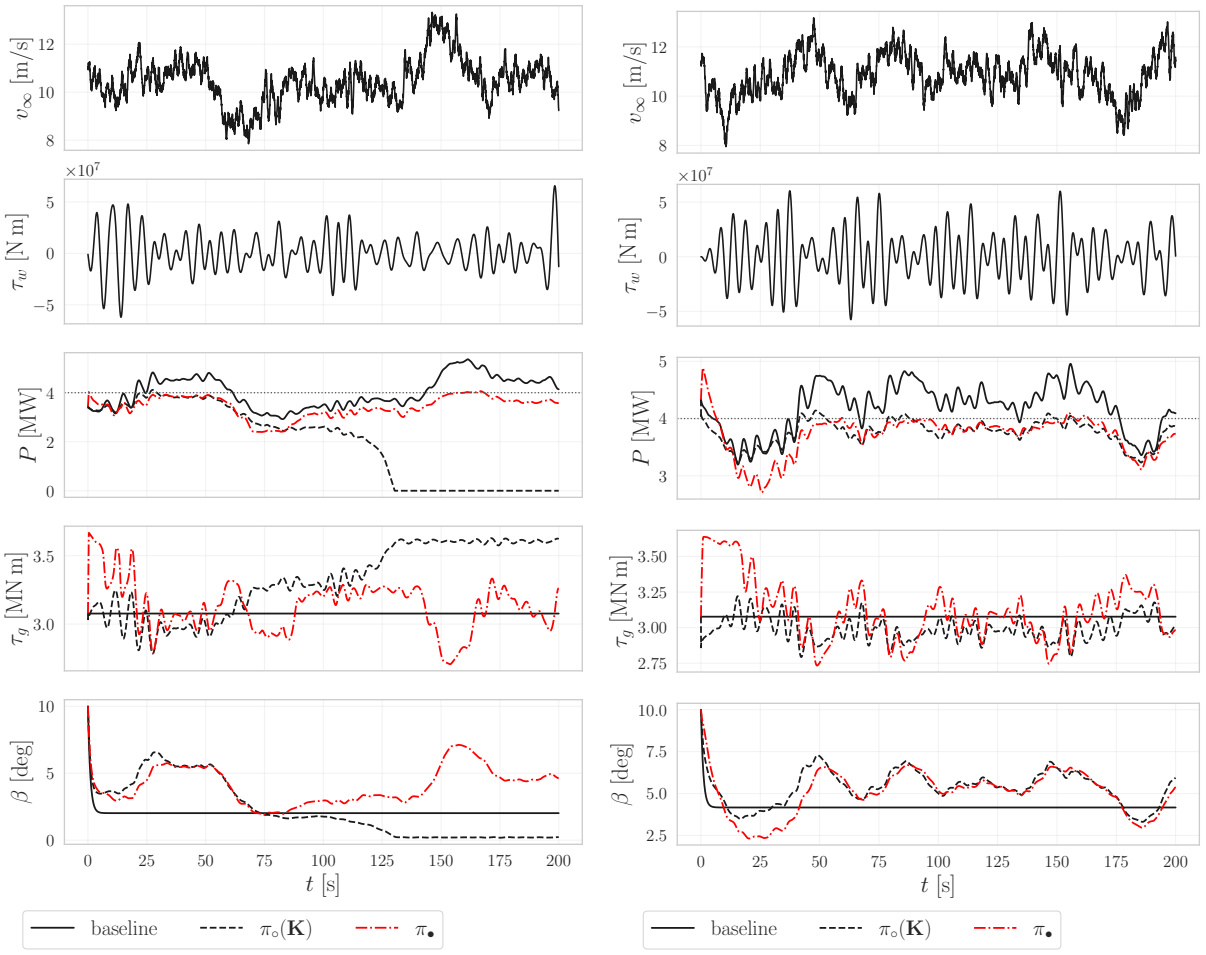


Figure 13: Aggregate policies performances over $M = 50$ episodes with randomized initial conditions and disturbances. Sensor noise is maintained identical to the training conditions. The RL policy is set in evaluation mode, e.g. we sample the target actor without any exploration noise on the selected action. Histogram represents the mean and bars are one standard deviation over the collected performances

relative performance of the learned model-free policy ($\pi_\bullet$) against the model-optimal ($\pi_\circ$) is investigated more in detail in Figure 13. This figure shows the performances of the two policies over a set of $M = 50$ evaluation episodes, subjecting the system to random initial conditions and stochastic wind and wave disturbances. Overall, the RL agent performs *on average* better than the model based-counterpart. In terms of the aggregate reward, which encapsulates both state regulation and actuator penalties, $\pi_\bullet$ not only achieves a higher (less negative) mean return but also reduces the standard deviation compared to the model-based expert, $\pi_\circ(\mathbf{K})$. This improved consistency is further confirmed by evaluating the mean absolute deviation from the smooth power target (Figure 13, right).

The physical rationale behind this performance improvement is shown in the time-domain simulations presented, for two cases, in Figure 14. The RL-based policy exhibits a twofold advantage. First, it applies a *local* policy improvement to optimize control trajectories in scenarios where the model-based controller struggles. As illustrated in Figure 14a, the tuned model-based policy, $\pi_\circ$, fails to maintain stable operation under these conditions, eventually shutting down the wind turbine completely at $t \approx 120$ s. Interestingly, the RL agent outperforms this 'master' policy by initially imitating its actions and then diverging when the model-based strategy becomes suboptimal. This transition

[1]Out of curiosity, the authors let it train for longer, and it reaches the baseline performance at episode ~ 1500.



(a) (b)

Figure 14: Comparison of control trajectories for model-based ($\pi_\circ(\boldsymbol{K})$) and model-free $\pi_\circ$ at the end of Phase 2. The RL-based policy manages to improve where the model-based struggled (Figure 14a), and (almost) converges to the available optimal solution when it is believed to be locally optimal (Figure 14b).

is evident in the β inputs shown in Figure 14a. Up to $t \approx 75$ s, the RL policy closely tracks $\pi_\circ$, but subsequently adapts its control law to keep the turbine near nominal operating conditions. This capability is further supported by the stable, on-design behavior depicted in Figure 14b, where $\pi_\circ$ and the RL policy, $\pi_\bullet$, achieve highly comparable performance. Ultimately, this demonstrates that the model-free agent successfully assimilated the fundamental control strategies of the model-based master while learning to surpass its limitations. A natural question is then whether these findings generalize when deployed to the "real-world" condition, which we investigate in the following section.

6.4 Phase 3: Deployment and mutual reinforcement

To evaluate the robustness and adaptability of the learned policies, this section transitions from the assimilated model used during initial training to the true plant, serving as our proxy for the real-world environment. Furthermore, the system is subjected to a broader

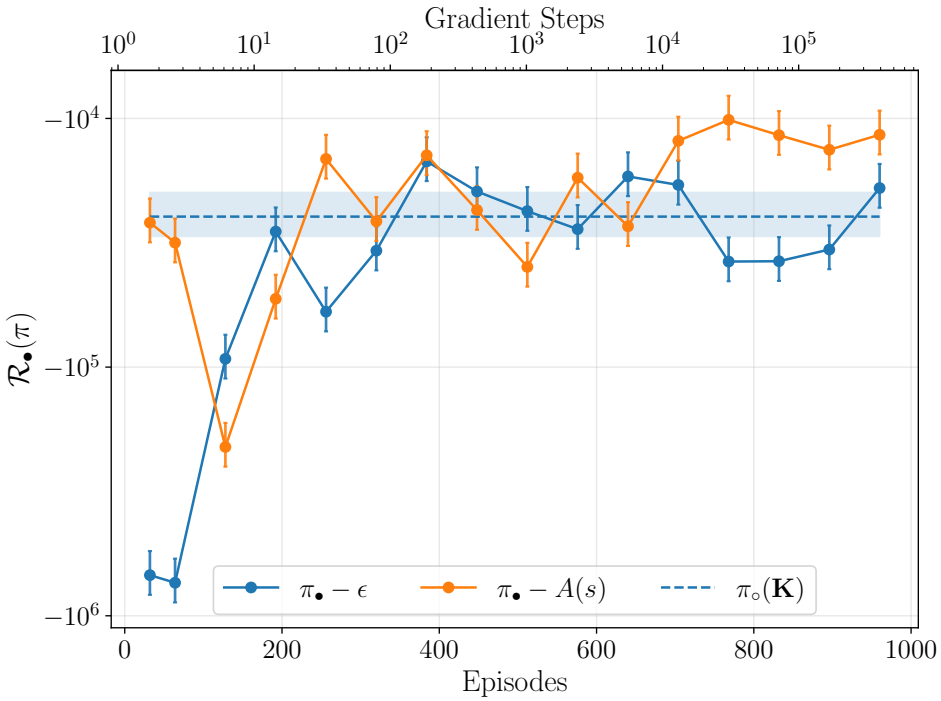


Figure 15: Learning curves of RL-based policies, deployed onto the real system and exposed to a wider disturbances set. The policy deployed with the epsilon greedy strategy is indicated as $\pi_\bullet - \epsilon$, while $\pi_\bullet - A(\boldsymbol{o}_\bullet)$ denotes the advantage-based policy switch. Each agent is evaluated for $M = 100$ episodes in deterministic mode (no noise over actions, sensor noise and random disturbances are kept as in training conditions).

spectrum of input disturbances, specifically wind speeds $\bar{v}_\infty \in [10, 15]$ m/s and wave periods $T_w \in [8, 12]$ s. The primary objective is to determine whether the model-free controller can successfully adapt to these off-design conditions, effectively demonstrating sim-to-real generalization across both unmodeled dynamics and expanded disturbance profiles. From a learning perspective, at this stage we reduce the exploration noise and increase the gradient steps per iteration to $24 \cdot 10^3$ and batch size to $B = 1024$. Further, the actor learning rate is reduced to slow down its response to possibly wrong signals arriving from the critic, which learning rate we increase to $\alpha_Q = 10^{-3}$.

To safely facilitate this transition and manage the exploration-exploitation trade-off, we investigate two distinct switching mechanisms between the model-based master ($\pi_\circ$) and the model-free agent ($\pi_\bullet$):

1. A time-decaying ϵ-greedy approach, wherein the model-based controller dictates the majority of actions during early interactions, gradually yielding control to the model-free policy over time. We consider $\epsilon = 0.3$ in this phase.

2. A value-based referee mechanism, which dynamically allocates control by evaluating the estimated advantage function. Specifically, the agent utilizes the learned action-value function (Critic) to compute the expected advantage of the model-free policy, $\pi_\bullet$, relative to the model-based baseline, $\pi_\circ$, at the current state s:

$$A(\boldsymbol{o}_\bullet) = Q(\boldsymbol{o}_\bullet, \pi_\bullet(\boldsymbol{o}_\bullet)) - Q(\boldsymbol{o}_\bullet, \pi_\circ(\boldsymbol{o}_\bullet))$$

Control is dynamically delegated to the model-free agent only when $A(\boldsymbol{o}_\bullet) > \Delta_Q$, ensuring that the transition occurs strictly when the model-free policy provides a mathematically positive expected advantage over the nominal controller. This approach directly descends from the Policy Improvement theorem, and we check this residual advantage every $\delta k = 40$ steps in the environment, delegating the control decisions to the chosen policy in the sub interval.

For these experiments, both switching algorithms are initialized using the best-performing agent weights obtained at the conclusion of Phase 2. The learning, or more specifically adaptation, metrics are shown in Figure 15. The early deployment stage is difficult for the model-free controller $\pi_\bullet$, regardless of the policy switch mechanism. The wider disturbance set and different system response require an initial phase of adaptation of the previously learned policy to the new environment. However, after $\sim$ 200 episodes, the model-free loop manages to improve the starting policy, with the $\pi_\bullet - A(\boldsymbol{o}_\bullet)$ switching method leading to better performance with respect to $\pi_\bullet - \epsilon$. It is now interesting to dwell

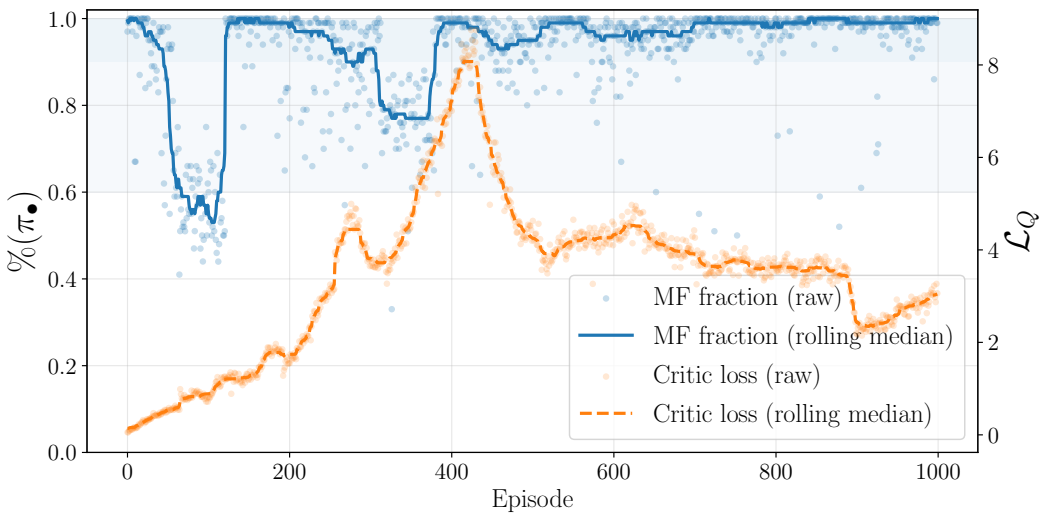
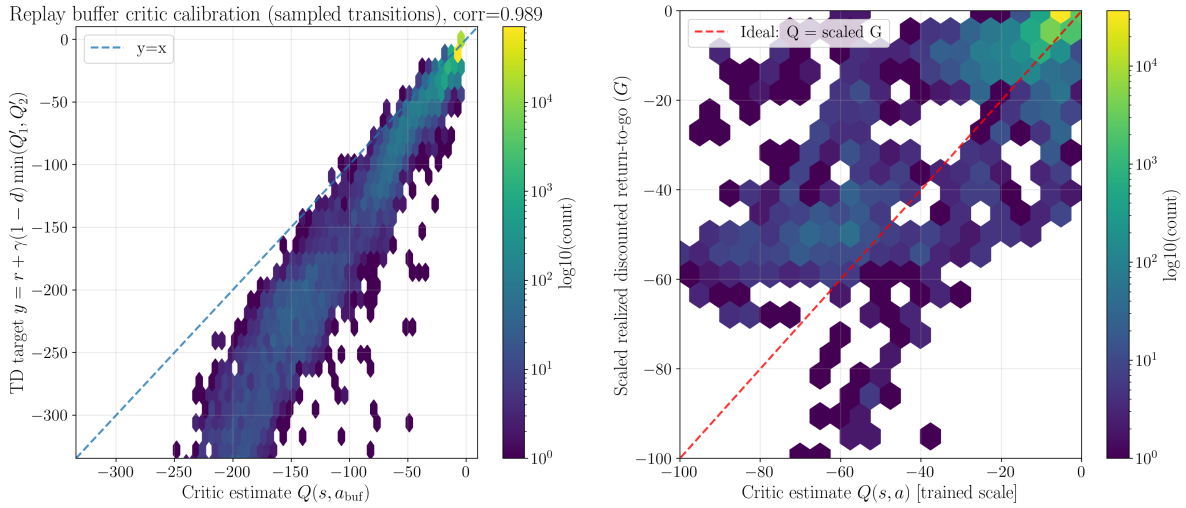


Figure 16: Superimposed advantage-based switching $A(\boldsymbol{o}_\bullet)$ and critic loss $\mathcal{L}_Q$ (Eq. (27)).

on the arbitration logic underlying $\pi_\bullet - A(\boldsymbol{o}_\bullet)$, and comparing the percentage of model-based/model-free control in driving the real system behavior. This analysis is shown in Figure 16. In the early deployment phase (episodes 0 to 200), the critic loss $\mathcal{L}_Q$ steadily increases as the replay buffer fills with transitions that do not correspond to what learned during Phase 2 (e.g. on the model). During this transient, the model-free fraction experiences a sharp drop, falling to approximately $\%(\pi_\bullet) \approx 60\%$. This indicates that the trained (and training) critic underestimates the value of the model-free policy, falling back to a 'safer' model-based alternative and leading to a highly mixed control strategy. A distinct regime can be obserbed between episodes 300 and 500. A massive spike in the critic loss indicates a period of high volatility and significant temporal-difference (TD) error, likely triggered by the agent discovering new, off-design state transitions that severely violate its previous value approximations. Strikingly, during and immediately following this period of critic instability, the gating mechanism overwhelmingly defaults to the Model-Free policy

($\%(\pi_\bullet) \approx 1.0$). This behavior persists in the rest of training phase, with the critic consistently choosing the model-free actor policy. The behavior of the critic is further studied in Figure 17. The Bellman consistency across the agent's sampled experience (from the



(a) (b)

Figure 17: In-depth analysis on the critic convergence at the end of the training phase for $\pi_\bullet - A(\boldsymbol{o}_\bullet)$. The critic self-consistency with the Temporal Difference (TD) error is shown in Figure 17a, while Figure 17b

replay buffer) is shown in Figure 17a. The tight aggregation of transitions along the $y = x$ diagonal (leading to a correlation of 0.989) indicates that the critic has successfully converged, capturing the local transition dynamics - at least for the most frequently visited states. Low-value regions of the state space, however, appear to be somewhat harder to assess, ultimately leading to residual TD error that indicates 'pessimism' of the critic in this region. Figure 17b evaluates the critic's absolute predictive accuracy by contrasting its point-in-time Q-estimates against the scaled, realized discounted return-to-go, defined as $G_t = \sum_{k=0}^{T} \gamma^k r_{t+k}$. The dense, perfectly calibrated cluster near the origin confirms that during nominal, steady-state operation, the physical system is highly stable, making future trajectories reliably deterministic and easy for the critic to predict. Conversely, as the system enters lower-reward regimes—indicative of severe transient disturbances or off-design failure modes—the realized returns exhibit massive variance. This dispersion highlights a fundamental characteristic of the continuous control task: while the critic correctly and consistently identifies these degraded states as heavily penalized (assigning appropriately negative Q-values), the exact cumulative penalty G_t is highly sensitive to the subsequent physical recovery trajectory.

We close this discussion evaluating the performance of the found model-free policy $\pi_\bullet$, relative to the baseline controller and model-based policy $\pi_\circ$.

The cumulative "real-world" (e.g. $\mathcal{R}_\bullet$) reward is consistently lower than the model-based counterpart, that systematically under-performs in the off-design condition and different dynamics underlying this Phase. This is supported by a reduction, almost by 50%, of the average power tracking error, showing in the righthand plot of Figure 18. We remind the reader that the sub-optimality of the adjoint-based optimal controller hinges

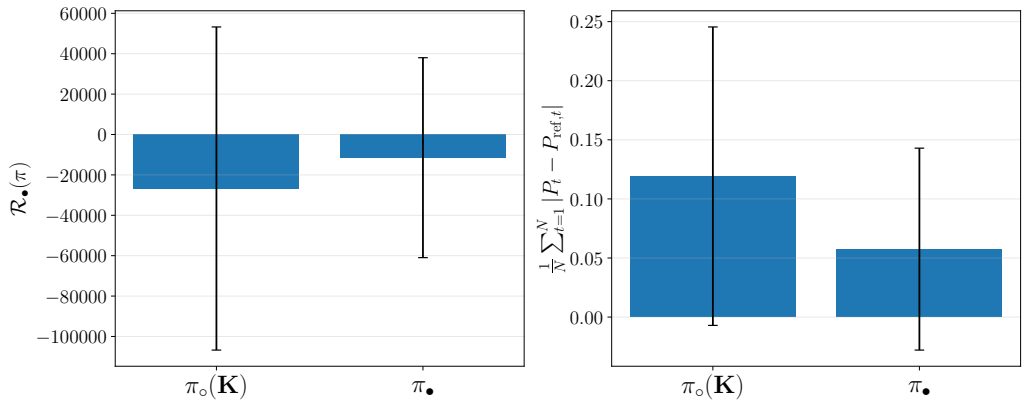


Figure 18: Aggregate policies performances over $M = 50$ episodes with randomized initial conditions and disturbances. Sensor noise is maintained identical to the training conditions. The RL policy is set in evaluation mode, e.g. we sample the target actor without any exploration noise on the selected action. Histogram represents the mean and bars are one standard deviation over the collected performances.

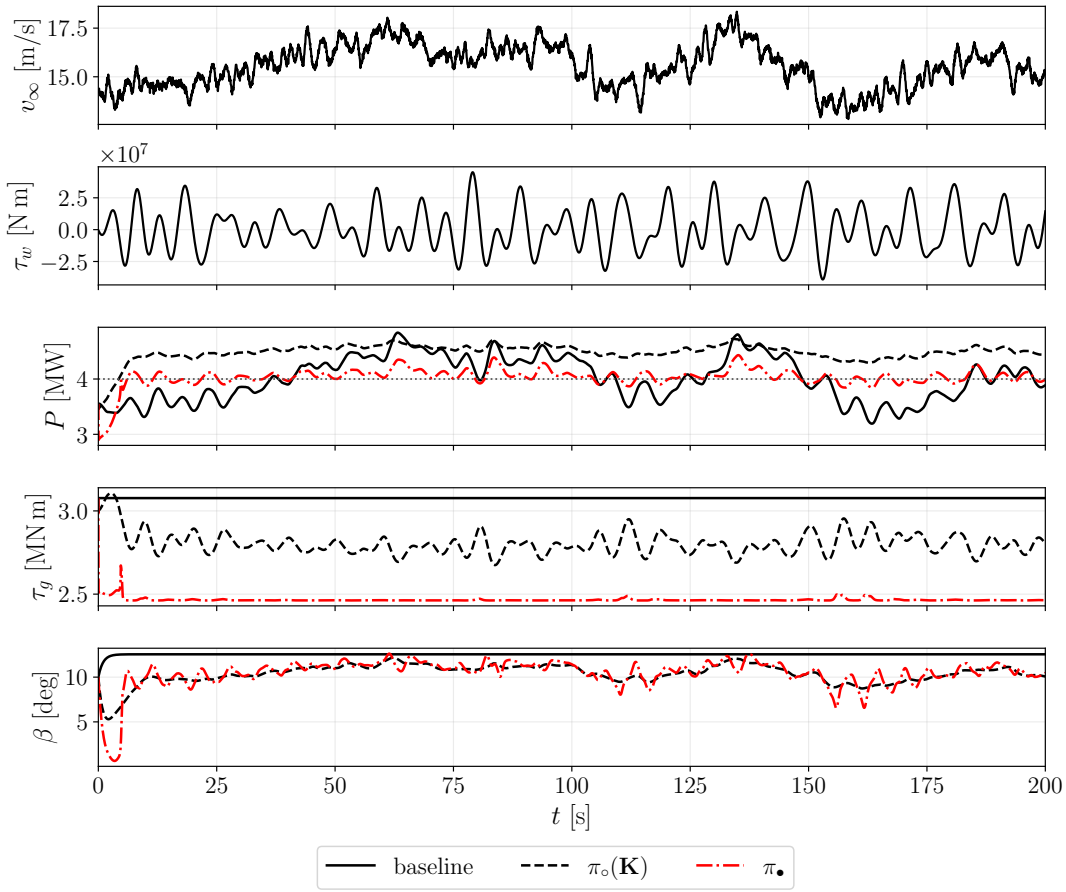


Figure 19: Comparison of control trajectories for model-based ($\pi_\circ(\mathbf{K})$) and model-free $\pi_\circ$ at the end of Phase 3, evaluated in the "real-world" environment and in *off-design* conditions.

on two main factors: (1) the real world dynamics is (slightly) different than the identified one (see Tab. 4); and (2) the controller in this phase operates in off-design conditions, e.g. both wind and wave sampled disturbances exhibit substantial variations with respect to the what considered in Phase 1.

The different strategy is shown in Figure 19. The final controller learns to delegate most of the regulation to the pitch actuation mechanism, despite the heavy penalty associated with its actuation. That is done by keeping the generator torque to an approximately constant value located close to the lower bound of the actuation space, while simultaneously introducing pseudo-periodic oscillations on top of the pitch command to shed the increased aerodynamic load in this high-wind regime.

In summary, the model-free policy successfully transcends the limitations of its model-based initialization. By interacting directly with the physical plant, the actor-critic algorithm maps out a compensatory strategy that explicitly accounts for unmodeled nonlinearities and off-design disturbances that the digital twin could not foresee. This phase definitively demonstrates the core value proposition of Reinforcement Twinning: the digital twin provides a safe, highly sample-efficient springboard for initial policy synthesis and bounds early exploration, while the data-driven RL loop provides the ultimate flexibility needed to adapt to the non-idealized realities of the physical world. The digital twin teaches the agent the foundational physics of stabilization; the real world teaches it how to master them.

7 Concluding Remarks and outlook

This chapter introduced Reinforcement Twinning (RT) as a unifying cyber–physical framework in which modeling and control are no longer treated as sequential tasks, but as mutually reinforcing processes. We began by revisiting the structural symmetry between adjoint-based model-based policy search and actor–critic reinforcement learning, emphasizing the deep connection between the Hamiltonian formulation of optimal control and the Bellman value formulation underlying Q-learning. By interpreting the state–action value function as a discrete-time counterpart of the Hamiltonian, we exposed a common variational backbone shared by both paradigms.

This symmetry was then elevated from a mathematical curiosity to an architectural principle. It motivated mechanisms of reciprocal reinforcement in which the digital twin improves the policy through structured prediction and sensitivity information, while the policy actively excites and probes the system to refine the twin. Within this view, learning is inherently bidirectional: control informs modeling, and modeling stabilizes and accelerates policy search. The framework naturally bridges modeling-to-control and control-to-modeling regimes and provides a coherent language to navigate between them.

The floating offshore wind turbine example served as a proof of concept, illustrating how identification, model-based control, and reinforcement learning can be embedded within a single architecture. The case study offered a concrete demonstration on how reciprocal reinforcement can be implemented in practice on a realistic nonlinear system subject to disturbances and uncertainty.

Several directions are currently under active development. A first priority is the experimental implementation of the RT framework across a diverse set of physical systems, in-

cluding floating wind turbines, aerial drones, and cryogenic storage units. These platforms span different time scales, actuation mechanisms, and dominant physical phenomena, yet share the same structural challenge: the need to reconcile limited models with data-driven adaptation under safety constraints. Bringing reciprocal reinforcement into laboratory and field experiments will test the robustness of the architecture beyond simulation and clarify how twin–policy co-evolution unfolds in real cyber–physical environments.

A second direction concerns high-dimensional flow-control problems, where the digital twin cannot be a full-order model but must rely on reduced-order modeling strategies. In this context, unresolved scales and closure terms become natural candidates for reciprocal adaptation. The control policy may not only act on the physical system but also inform the calibration of reduced-order dynamics, while the twin provides structured low-dimensional predictions that guide exploration in a tractable subspace. Extending RT to such settings would open a path toward hybrid closure modeling in which data-driven reinforcement complements physics-based reduction.

Finally, an alternative formulation is being explored in which the reciprocal mechanism is made gradient-free and probabilistic. Instead of relying on adjoint sensitivities or policy gradients, the twin and policy may be coupled through multifidelity probabilistic models, akin to Gaussian process regression frameworks. In this perspective, the twin encodes structured prior knowledge and uncertainty, while the policy operates as an adaptive query mechanism that selects informative experiments. Reciprocal reinforcement then emerges through Bayesian updating and uncertainty reduction rather than through explicit gradient exchange.

Reinforcement Twinning is still in its infancy and offers an organizing principle for the co-adaptation of models and policies. Whether implemented through gradients, reduced-order closures, or probabilistic surrogates, the central idea remains the same: the digital twin and the policy should learn together, in continuous dialogue with the real system, as engineering transitions from static modeling to adaptive and active representations in the age of data.

References

Abarbanel, H. D. I., Rozdeba, P. J., and Shirman, S. (2018). Machine learning in data assimilation for chaotic dynamical systems. *Neural Computation*, 30(7):1811–1862.

Abbas, N. J., Zalkind, D. S., Pao, L., and Wright, A. (2022). A reference open-source controller for fixed and floating offshore wind turbines. *Wind Energy Science*, 7(1):53–73.

Adler, E. J. and Martins, J. R. (2023). Hydrogen-powered aircraft: Fundamental concepts, key technologies, and environmental impacts. *Progress in Aerospace Sciences*, 141:100922.

Anderson and Johnstone (1985). Global adaptive pole positioning. *IEEE Transactions on Automatic Control*.

Ariyur, K. B. and Krstic, M. (2003). *Real-Time Optimization by Extremum-Seeking Control*. John Wiley & Sons.

Asch, M., Bocquet, M., and Nodet, M. (2016). *Data Assimilation: Methods, Algorithms, and Applications*. SIAM.

Åström, K. J. and Wittenmark, B. (1995). *Adaptive Control*. Addison-Wesley, 2nd edition.

Atzori, L., Iera, A., and Morabito, G. (2010). The internet of things: A survey. *Computer Networks*, 54(15):2787–2805.

Badwe, A. S., Gudi, R. D., Patwardhan, S. C., and Shah, S. L. (2010). Quantifying the impact of model-plant mismatch on controller performance. *Journal of Process Control*, 20(4):408–425.

Bansal, S., Calandra, R., Xiao, T., and Levine, S. (2017). Mbmf: Model-based priors for model-free reinforcement learning. In *CoRL*.

Bardi, M. and Capuzzo-Dolcetta, I. (1997). *Optimal Control and Viscosity Solutions of Hamilton-Jacobi-Bellman Equations*. Birkhäuser.

Bernhardsson, B. (1989). Dual control of a first-order system with two possible gains. *International Journal of Adaptive Control and Signal Processing*, 3(1):15–22.

Bertsekas, D. P. (2005). Dynamic Programming and Suboptimal Control: A Survey from ADP to MPC*. *European Journal of Control*, 11(4-5):310–334.

Blanke, M., Kinnaert, M., Lunze, J., and Staroswiecki, M. (2016). *Diagnosis and Fault-Tolerant Control*. Springer, 3rd edition.

Buckman, J., Hafner, D., Tucker, G., Brevdo, E., and Lee, H. (2018). Sample-efficient reinforcement learning with stochastic ensemble value expansion. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 8224–8234.

Buizza, R. et al. (2022). Data learning dynamics for model reduction. *Physical Review Fluids*.

Carnarius, A., Thiele, F., Özkaya, E., and Gauger, N. R. (2011). Optimal control of the cylinder wake in the laminar regime by trust-region evolution strategies. *Journal of Computational Physics*, 230(9):3348–3364.

Chaaban, R. and C-P., F. (2014). Reducing blade fatigue and damping platform motions of floating wind turbines using model predictive control. *International Conference on Structural Dynamics*.

Chen, W.-H., Rhodes, C., and Liu, C. (2021). Dual Control for Exploitation and Exploration (DCEE) in Autonomous Search.

Chiuso, A. and Pillonetto, G. (2019). System identification: A machine learning perspective. *Annual Review of Control, Robotics, and Autonomous Systems*, 2:281–304.

Clavera, I., Fu, J., and Abbeel, P. (2018). Model-based reinforcement learning via meta-policy optimization. In *Proceedings of the 2nd Conference on Robot Learning (CoRL)*, pages 617–629.

Dehkordi, M. B., Forgione, M., and Piga, D. (2025). Uncertainty quantification in neural state-space models: Applications for experiment design and uncertainty-aware MPC. *European Journal of Control*, 85:101359.

Dixon, L. C. W. (1981). The adjoint method for sensitivity analysis of dynamic systems. *Numerical Optimization of Dynamic Systems*.

Feinberg, V., Wan, A., Stoica, I., Jordan, M. I., Gonzalez, J. E., and Levine, S. (2018). Model-based value estimation for efficient model-free reinforcement learning. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 1469–1478.

Feldbaum, A. A. (1960). Dual control theory. *Automation and Remote Control*, 21:874–880.

Feldbâum, A. A. (1963). Dual control theory problems. *IFAC Proceedings Volumes*, 1(2):541–550.

Ficili, I., Giacobbe, M., Tricomi, G., and Puliafito, A. (2025). From sensors to data intelligence: Leveraging iot, cloud, and edge computing with ai. *Sensors*, 25(6):1763.

Fujimoto, S., Hoof, H., and Meger, D. (2018). Addressing function approximation error in actor-critic methods. In *International conference on machine learning*, pages 1587–1596. PMLR.

Ghiglieri, J. and Ulbrich, M. (2014). Optimal control of PDEs with application to flow control. *Optimization and Engineering*.

Goodwin, G. C. and Payne, R. L. (1977). *Dynamic System Identification: Experiment Design and Data Analysis*. Academic Press.

Grondman, I., Busoniu, L., Lopes, G. A. D., and Babuska, R. (2012). A Survey of Actor-Critic Reinforcement Learning: Standard and Natural Policy Gradients. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, 42(6):1291–1307.

Gubbi, J., Buyya, R., Marusic, S., and Palaniswami, M. (2013). Internet of things (iot): A vision, architectural elements, and future directions. *Future Generation Computer Systems*, 29(7):1645–1660.

Gunzburger, M. D. (2002). *Perspectives in Flow Control and Optimization*. SIAM.

Ioannou, P. A. and Sun, J. (1996). *Robust Adaptive Control*. Prentice Hall.

Isermann, R. (2006). *Fault-Diagnosis Systems: An Introduction from Fault Detection to Fault Tolerance*. Springer.

Johannink, T., Bahl, S., Nair, A., Luo, J., Kumar, A., Loskyll, M., Ojea, J. A., Solowjow, E., and Levine, S. (2019a). Residual reinforcement learning for robot control. In *2019 International Conference on Robotics and Automation (ICRA)*, pages 6023–6029. IEEE.

Johannink, T. et al. (2019b). Residual reinforcement learning for robot control. In *ICRA*.

Jones, D., Snider, C., Nassehi, A., Yon, J., and Hicks, B. (2020). Characterising the digital twin: A systematic literature review. *CIRP Journal of Manufacturing Science and Technology*, 29:36–52.

Jonkman, J., Butterfield, S., Musial, W., and Scott, G. (2009). Definition of a 5-mw reference wind turbine for offshore system development. Technical report, National Renewable Energy Laboratory (NREL), Golden, CO.

Jonkman, J. M. (2007). Dynamics modeling and loads analysis of an offshore floating wind turbine. In *Proceedings of the OMAE 2007 Conference*.

Kagermann, H., Wahlster, W., and Helbig, J. (2013). Recommendations for implementing the strategic initiative industrie 4.0. Final report of the industrie 4.0 working group, National Academy of Science and Engineering.

Kahn, G., Villaflor, A., Pong, V., Abbeel, P., and Levine, S. (2017). Uncertainty-Aware Reinforcement Learning for Collision Avoidance.

Karikomi, K. et al. (2015). Wind tunnel testing on negative-damped responses of a 7mw floating offshore wind turbine. In *EWEA 2015*.

Korenhof, P., Blok, V., and Kloppenburg, S. (2021). Steering representations—towards a critical understanding of digital twins. *Philosophy & Technology*, 34(4):1751–1773.

Krstić, M. (2000). Performance improvement and limitations in extremum seeking control. *Systems & Control Letters*, 39(5):313–326.

Kumar, V. and Michael, N. (2012). Opportunities and challenges with autonomous micro aerial vehicles. *The International Journal of Robotics Research*, 31(11):1279–1291.

Lackner, M. A. (2009). Controlling platform motions and reducing blade loads for floating wind turbines. *Wind Engineering*, 33(6):541–553.

Larsen, T. J. and Hanson, T. D. (2007). A method to avoid negative damped low frequent tower vibrations for a floating, pitch controlled wind turbine. In *Journal of Physics: Conference Series*, volume 75. IOP.

Lee, E. A. (2008). Cyber physical systems: Design challenges. In *2008 11th IEEE International Symposium on Object and Component-Oriented Real-Time Distributed Computing*, pages 363–369.

Lee, E. A. and Seshia, S. A. (2015). *Introduction to Embedded Systems: A Cyber-Physical Systems Approach*. MIT Press.

Li, X. and Gao, H. (2015). Load mitigation for a floating wind turbine via generalized h_∞ structural control. *IEEE Transactions on Industrial Electronics*, 63(1):332–342.

Ljung, L. (1999). *System Identification: Theory for the User*. Prentice Hall, 2nd edition.

Marques, P. A., Ahizi, S., and Mendez, M. A. (2024). Real-time data assimilation for the thermodynamic modeling of cryogenic storage tanks. *Energy*, 302:131739.

Mazzaretto, O. M., Menéndez, M., and Lobeto, H. (2022). A global evaluation of the jonswap spectra suitability on coastal areas. *Ocean Engineering*, 266:112756.

Mendez, M. A., Berghe, J. v. D., Ratz, M., Fiore, M., and Schena, L. (2025). Learning with physical constraints. Chapter 3 from "from Machine Learning for Fluid Dynamics (ISBN 978-2875162090).".

Mesbah, A. (2018). Stochastic model predictive control with active uncertainty learning: A Survey on dual control. *Annual Reviews in Control*, 45:107–117.

Moerland, T. M., Broekens, J., and Jonker, C. M. (2023). Model-based reinforcement learning: A survey. *Foundations and Trends in Machine Learning*, 16(1):1–118. (Preprint 2022).

Moore, J., Ryall, T., and Xia, L. (1989). Central tendency adaptive pole assignment. *IEEE Transactions on Automatic Control*, 34(3):363–367.

Nagabandi, A. et al. (2018). Neural network dynamics for model-based deep reinforcement learning with model-free fine-tuning. In *ICRA*.

Namik, H., Rotea, M., and Lackner, M. (2013). Active structural control with actuator dynamics on a floating wind turbine. In *51st AIAA Aerospace Sciences Meeting*, page 455.

Pathak, D., Agrawal, P., Efros, A. A., and Darrell, T. (2017). Curiosity-driven exploration by self-supervised prediction. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*.

Peng, S. (1992). Stochastic Hamilton-Jacobi-Bellman equations. *SIAM Journal on Control and Optimization*, 30(2):284–304.

Pino, F. et al. (2023). Active flow control for drag reduction through multi-agent reinforcement learning on a turbulent cylinder. *Physics of Fluids*.

Poletti, R., Schena, L., Koloszar, L., Degroote, J., and Mendez, M. A. (2025). Reinforcement twinning for hybrid control of flapping-wing drones.

Polydoros, A. S. and Nalpantidis, L. (2017). Survey of model-based reinforcement learning: Applications on robotics. *Journal of Intelligent & Robotic Systems*, 86:153–173.

Randino, S., Schena, L., Coudou, N., Garone, E., and Mendez, M. A. (2025). Nonlinear system identification for model-based control of waked wind turbines. *arXiv preprint arXiv:2510.07336*.

Ravindran, S. S. (2000). Reduced-order modeling of approximations to the Navier-Stokes equations. *Computers & Mathematics with Applications*, 40(10-11):1319–1334.

Robertson, A., Jonkman, J., Masciola, M., Song, H., Goupee, A., Coulling, A., and Luan, C. (2014). Definition of the semisubmersible floating system for phase ii of oc4. Technical report, National Renewable Energy Laboratory (NREL), Golden, CO.

Rowley, C. W. and Dawson, S. T. M. (2017). Model reduction for flow analysis and control. *Annual Review of Fluid Mechanics*, 49:387–417.

Ryall, T. and Moore, J. (1989). Central tendency minimum variance adaptive control. *IEEE Transactions on Automatic Control*, 34(3):367–371.

Schaul, T., Quan, J., Antonoglou, I., and Silver, D. (2015). Prioritized experience replay.

Schena, L., Marques, P. A., Poletti, R., Ahizi, S., Van den Berghe, J., and Mendez, M. A. (2024). Reinforcement twinning: From digital twins to model-based reinforcement learning. *Journal of Computational Science*, 82:102421.

Schmidhuber, J. (2010). Formal theory of creativity, fun, and intrinsic motivation. *IEEE Transactions on Autonomous Mental Development*, 2(3):230–247.

Schwenzer, M., Ay, M., Bergs, T., and Abel, D. (2021). Review on model predictive control: an engineering perspective. *The International Journal of Advanced Manufacturing Technology*, 117:1327–1349.

Sicari, S., Rizzardi, A., Grieco, L. A., and Coen-Porisini, A. (2015). Security, privacy and trust in internet of things: The road ahead. *Computer Networks*, 76:146–164.

Silver, D., Lever, G., Heess, N., Degris, T., Wierstra, D., and Riedmiller, M. (2014). Deterministic policy gradient algorithms. In *Proceedings of the 31st International Conference on Machine Learning (ICML)*, volume 32 of *Proceedings of Machine Learning Research*, pages 387–395.

Sritharan, S. S., editor (1998). *Optimal Control of Viscous Flow*. SIAM.

Stengel, R. F. (1994). *Optimal Control and Estimation*. Dover Publications.

Stockhouse, D., Phadnis, M., Grant, E., Johnson, K., Damiani, R., and Pao, L. (2022). Control of a floating wind turbine on a novel actuated platform. In *2022 American Control Conference (ACC)*, pages 3532–3537.

Sutton, R. S. (1991). Dyna, an integrated architecture for learning, planning, and reacting. In *ACM SIGART Bulletin*, volume 2, pages 160–163.

Sutton, R. S. and Barto, A. G. (1998). *Reinforcement Learning: An Introduction*. MIT press.

Tao, F., Qi, Q., Liu, A., and Nee, A. Y. C. (2019). Digital twins and cyber–physical systems toward smart manufacturing and industry 4.0: Correlation and comparison. *Engineering*, 5(4):653–661.

Torabi, F., Warnell, G., and Stone, P. (2018). Behavioral cloning from observation. *arXiv preprint arXiv:1805.01954*.

Veers, P., Dykes, K., Lantz, E., Barth, S., Bottasso, C. L., Carlson, O., Clifton, A., Green, J., Green, P., Holttinen, H., Laird, D., Lehtomäki, V., Lundquist, J. K., Manwell, J., Marquis, M., Meneveau, C., Moriarty, P., Munduate, X., Muskulus, M., Naughton, J., Pao, L., Paquette, J., Peinke, J., Robertson, A., Sanz Rodrigo, J., Sempreviva, A. M., Smith, J. C., Tuohy, A., and Wiser, R. (2019). Grand challenges in the science of wind energy. *Science*, 366(6464).

Wagg, D. J., Burr, C., Shepherd, J., Xuereb Conti, Z., Enzer, M., and Niederer, S. (2025). The philosophical foundations of digital twinning. *Data-Centric Engineering*, 6.

Werner, R. et al. (2023). Deep reinforcement learning for flow control exploits different physics for increasing Reynolds number regimes. *Fluids*.

Wittenmark, B. (1995). Adaptive Dual Control Methods: An Overview. *IFAC Proceedings Volumes*, 28(13):67–72.

Wittenmark, B. and Elevitch, C. (1985). An Adaptive Control Algorithm with Dual Features. *IFAC Proceedings Volumes*, 18(5):587–592.

Åström, K. J. and Helmersson, A. (1986). Dual control of an integrator with unknown gain. *Computers & Mathematics with Applications*, 12(6, Part A):653–662.

Åström, K. J. and Wittenmark, B. (2008). *Adaptive Control*. Dover Publications.